



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

PixelProbe - AI Image Detection System

Naoise O'Loughlin

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Acknowledgements

I would like to thank my supervisor, Dr Fergus Downey, for all his help and guidance throughout this project. I would also like to thank my family, friends and my partner Louise for all their support throughout the year.

List of Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DOM	Document Object Model
GAN	Generative Adversarial Network
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identification
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML
JWT	JSON Web Token
LLM	Large Language Model
MHSA	Multi-Head Self Attention
ML	Machine Learning
MUI	Material UI
NPM	Node Package Manager
OAuth	Open Authentication
ORM	Object-Relational Mapping
S3	Amazon Simple Storage Service
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource

Summary

PixelProbe is a full-stack web application with the aim of aiding the user in the classification of Real-life images vs AI-generated images. It implements many of the skills and knowledge obtained throughout the author’s 4 years of study in ATU Galway. The project back end is written in Python utilising the TensorFlow/Keras libraries, with a ReactJS frontend written in JSX, JavaScript and CSS.

The frontend is hosted locally through a NodeJS server, while the back end is hosted on an AWS instance.

Key features include a real-time image classification interface, scalable API for model inference, EfficientNet transfer learning pipeline, persistent storage & user data management, secure user authentication, and Cloud deployment & storage integration.

The user uploads an image on the React site, which is then passed to the FastAPI back end, which hosts the model inference pipeline endpoints. This image is fed to the model, and inference is completed, returning a prediction output along side the model’s confidence level. The personalised user upload history can then be viewed on the website with the prediction, confidence level, and timestamp included.

All information is stored in an SQLite database, which includes the prediction, confidence level, timestamp, the image name, and the user who uploaded. All images are saved into an S3 bucket hosted on an AWS instance.

This project was an overall success, with all original functionality proposed successfully implemented and demonstrated.

Contents

Declaration	i
Acknowledgements	ii
List of Acronyms	iii
Summary	iv
1 Introduction	1
1.1 Motivation	1
1.2 Project Goals	2
1.3 Report Overview	2
2 Project Architecture	3
2.1 Architecture Overview	4
2.2 Back end - FastAPI	4
2.2.1 Why FastAPI?	4
2.2.2 Structure	4
2.2.3 API Design	6
2.2.4 Request/Response Models (Pydantic)	8
2.2.5 Authentication & Security (JWT)	9
2.2.6 Model Integration	9
2.2.7 SQLite	10
2.3 Frontend - React	11
2.3.1 Technology Stack	11
2.3.2 Component Structure	12
2.3.3 State Management	13
2.3.4 API Communication	13
2.3.5 Routing and Navigation	13
2.3.6 Security	14
2.4 Database - SQLite	14
2.4.1 Schema	15
2.4.2 ORM Integration (SQLAlchemy)	15
2.4.3 Database Operations	16
2.4.4 Data persistence and Storage	16
2.4.5 Security	16
2.4.6 Future Improvements	17
2.5 Cloud Architecture - S3	17
2.5.1 Cloud Overview	17
2.5.2 S3 Storage	17
2.5.3 Back end Hosting Environment	18
2.5.4 Security	18
2.5.5 Scalability and Future Improvements	18

3	Machine Learning	19
3.1	Datasets	19
3.2	Model Architecture	19
3.3	Training Process	20
3.3.1	Phase 1 - Train Classification Head only	20
3.3.2	Phase 2 - Fine-Tuning	20
3.3.3	Callbacks	20
3.4	Evaluation	20
3.4.1	Accuracy	20
3.4.2	Confusion Matrix	21
3.4.3	Output and Storage	21
3.5	Results	21
3.5.1	Training Behaviour	21
3.5.2	Test Set Performance	21
3.5.3	Cross-Dataset Generalisation	22
3.5.4	Overall Assessment	22
4	Implementation & Results	23
4.1	Back end	23
4.1.1	Structure	23
4.1.2	FastAPI	23
4.1.3	Security	30
4.2	Frontend	33
4.2.1	Structure	33
4.2.2	Authentication	34
4.2.3	Image Upload	35
4.2.4	Prediction History	36
4.2.5	User Interface (UI)	37
4.2.6	API integration	38
4.3	Machine Learning	39
4.3.1	Dataset Preparation	39
4.3.2	Preprocessing and Data Augmentation	41
4.3.3	Model Architecture	42
4.3.4	Training Pipeline	45
4.3.5	Model Export and Integration	47
4.3.6	Evaluation	47
4.3.7	Overall Summary	49
4.4	Results	49
4.4.1	Training and Validation Performance	49
4.4.2	Test Accuracy	51
4.4.3	Confusion Matrix Analysis	51
4.4.4	Cross-Dataset Comparison	52
4.4.5	Summary of Results	52
5	Conclusion	53
6	Future Work	54
6.1	Video Frame-by-Frame Detection	54
6.2	Mobile App Deployment	54
6.3	Summary	54

A	Appendix - Additional Material	55
A.1	Project Poster	55
A.2	Training Curves & Confusion Matrices	56
A.3	LiteMHSA Class code	58

List of Figures

2.1	Project Architecture	3
2.2	System Structure	4
3.1	Model Architecture	19
4.1	Back end Folder Structure	23
4.2	<code>main.py</code> showing FastAPI app setup, middleware, and router registration.	24
4.3	SQLAlchemy ORM models for users and predictions in <code>models.py</code>	25
4.4	<code>classify_image</code> helper used for model inference.	26
4.5	S3 client configuration in the FastAPI back end.	26
4.6	Implementation of the <code>/api/predictions</code> endpoint in <code>main.py</code>	27
4.7	Full implementation of the <code>/api/uploadfile</code> endpoint in <code>main.py</code>	29
4.8	JWT-based authentication and <code>get_current_user</code> implementation in the <code>users</code> module.	29
4.9	Frontend File Structure	33
4.10	PixelProbe Login Screen	35
4.11	Image upload interface	36
4.12	Prediction History Interface	37
4.13	Main UI Screen	38
4.14	Internal structure of the LiteMHSA attention mechanism.	43
4.15	Complete model architecture	44
4.16	Training and validation loss across all epochs.	50
4.17	Training and validation accuracy across all epochs.	50
4.18	Confusion matrix for one of the trained model versions.	51
A.1	B1_V3 Training curves	56
A.2	B1_V3 Confusion Matrix	56
A.3	B1_CFV2 Training curves	56
A.4	B1_CFV2 Confusion Matrix	56
A.5	B1_GI Training curves	56
A.6	B1_GI Confusion Matrix	56
A.7	B2_GI Training curves	57
A.8	B2_GI Confusion Matrix	57
A.9	B2_GI_v3 Training curves	57
A.10	B2_GI_v3 Confusion Matrix	57
A.11	Full implementation of the LiteMHSA layer referenced in Section 4.3.3	58

1 Introduction

The rise of Large Language Models (LLMs) such as Claude, ChatGPT and Copilot has had a huge effect on the modern landscape of the world both technically and socially, having a huge impact on public communications, productivity, and information access across the globe.

In parallel, the development of generative image platforms such as DALL-E, Midjourney, and Stable Diffusion has had similar impacts, allowing users to generate photorealistic imagery as well as edit existing photos based on user prompts. As these models continue to progress, the distinction between real and generated images continues to blur, leading to an increased challenge of maintaining public trust and the authenticity of online information. [1]

Incidents such as the recent Grok controversy in 2025 have brought more attention to the misuse of these platforms. Some researchers estimated that up to 6700 inappropriate images were being produced per hour on Grok’s Imagine model, leading to large-scale investigations into the usage of these types of generative models. [2]

1.1 Motivation

The advancements of generative AI have introduced both benefits and challenges in the fight against misinformation online. On one hand, LLMs and image recognition models can be trained to distinguish AI generated media from real articles and detect subtle patterns that humans miss; on the other side, it is also capable of being trained to produce the same visual and text based media with increasingly realistic results.

A current issue is the asymmetry between media generation and detection. Over the past few years, generation has become very straightforward and accessible to all, with a range of different generation methods such as GANs (Generative Adversarial Networks), diffusion models, and transformer based systems that can all create photorealistic images and clear text. This diversity in generation models has brought challenges to detection systems, models trained to identify the artifacts common in one generative model often fail to generalise to others, leading to reduced reliability on unseen generators and data. [3]

This expanding gap between generation and detection has significant social impacts. As synthetic media become more accessible and increasingly realistic, the potential to spread misinformation also increases. Ensuring authenticity of online content is quickly becoming a critical priority, this demands a deeper understanding of the common features and differences across generator types. To keep up with rapidly evolving generators, it is essential for detection systems to have the ability to adapt as quickly, and develop methods that generalise across model families, remaining robust as generative AI advances

1.2 Project Goals

The goals of this project are as follows.

- Develop a full-stack application for real vs AI generated image classification.
- FastAPI back end dealing with model loading and inference
- Database to store all historical data i.e previous predictions and uploaded images
- React front-end to allow user uploads and display previous predictions.
- Generalisation of the model’s classification on unseen data.

This project is designed to be scalable and implement additional features to what is outlined above as key goals.

1.3 Report Overview

This report provides a complete account of the design, development and evaluation of the PixelProbe AI image detection system. Chapter 1 shows the motivation behind the project, an outline of the challenges posed by modern AI generators, and defines project goals

Chapter 2 covers the overall system architecture, detailing the FastAPI back end, model integration pipeline, database design, storage and React frontend. The following chapters cover implementation of each component within the project, the machine learning development process, and the training and evaluation process used to assess model performance.

The report also contains a discussion of results, system limitations and potential future enhancements. Throughout the document, diagrams, API specifications, and referenced literature are provided to support technical explanations.

2 Project Architecture

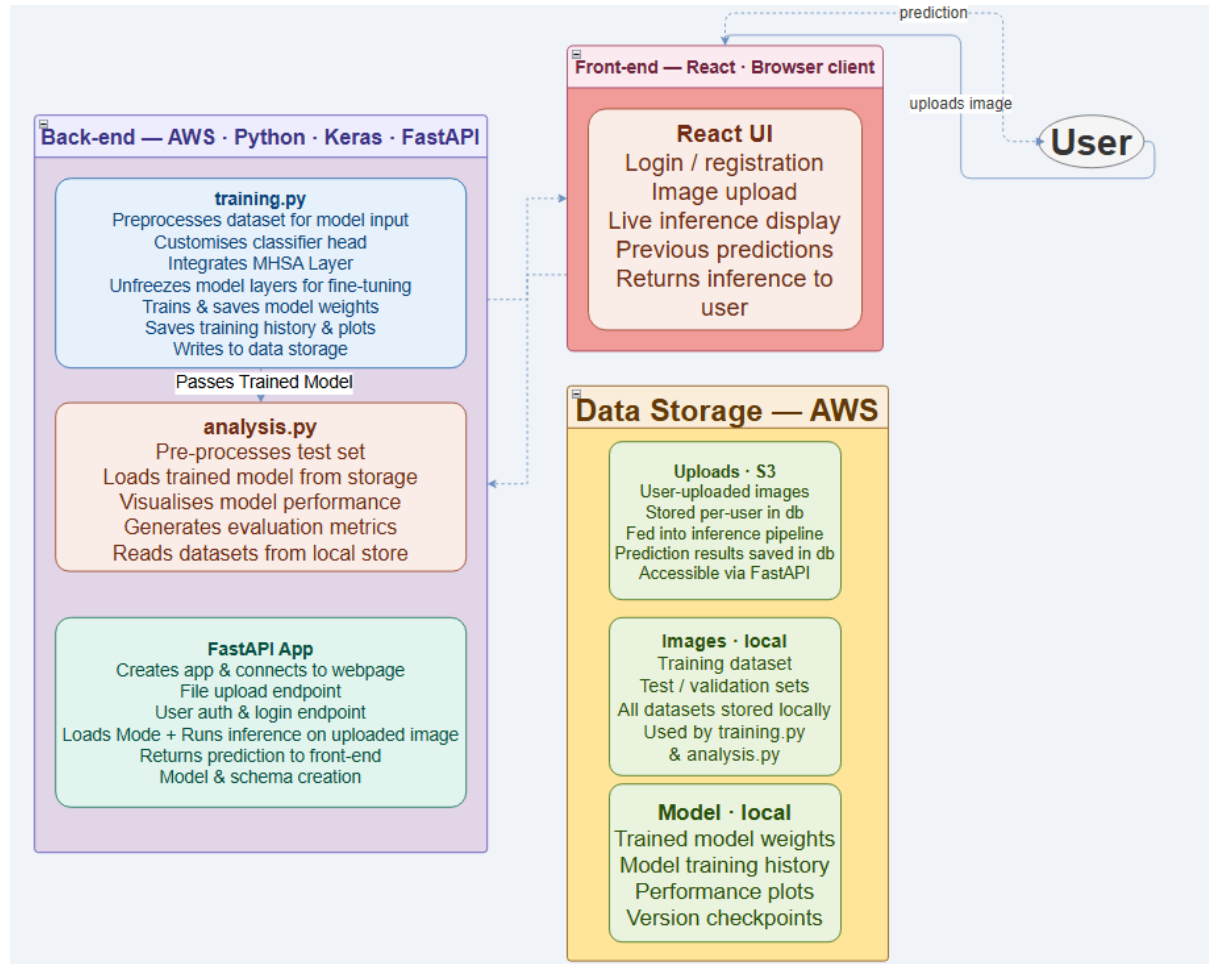


Figure 2.1: Project Architecture

2.1 Architecture Overview

This system enables users to interact with a React-based web application, uploading images from their device, and receive real-time authenticity classification results. The React frontend communicates with a fastAPI back end written in Python, which loads a trained inference model at startup and exposes prediction endpoints to the client.

User authentication is supported through registration and login functionality, which allows each user to maintain a personalised prediction history. All model outputs, and user-specific prediction records are stored in an SQLite database, while uploaded images are stored in an Amazon S3 bucket. This architecture ensures efficient inference, persistent user history and scalable secure image storage.

2.2 Back end - FastAPI

2.2.1 Why FastAPI?

FastAPI is a modern, high performance web framework designed for building Python-based APIs. It offers intuitive development, editor support, and extensive documentation, making it well-suited for a project of this scale. It is built on the OpenAPI standard (formerly Swagger) and JSON schema, it automatically generates interactive API documentation, leading to clear communication and easy integration with React frontend [4]

Its asynchronous request handling allows the back end to manage multiple image-upload and inference requests in parallel. When deployed using an ASGI server such as Uvicorn, FastAPI can effectively handle high-concurrency workloads while maintaining low latency responses. FastAPI allows Tensorflow/Keras models to be loaded once at startup and continuously reused across all requests, which supports real-time classification. This makes FastAPI a good fit for this project allowing the deployment of lightweight, and scalable ML inference services with web-based architectures. [5] [6]

2.2.2 Structure

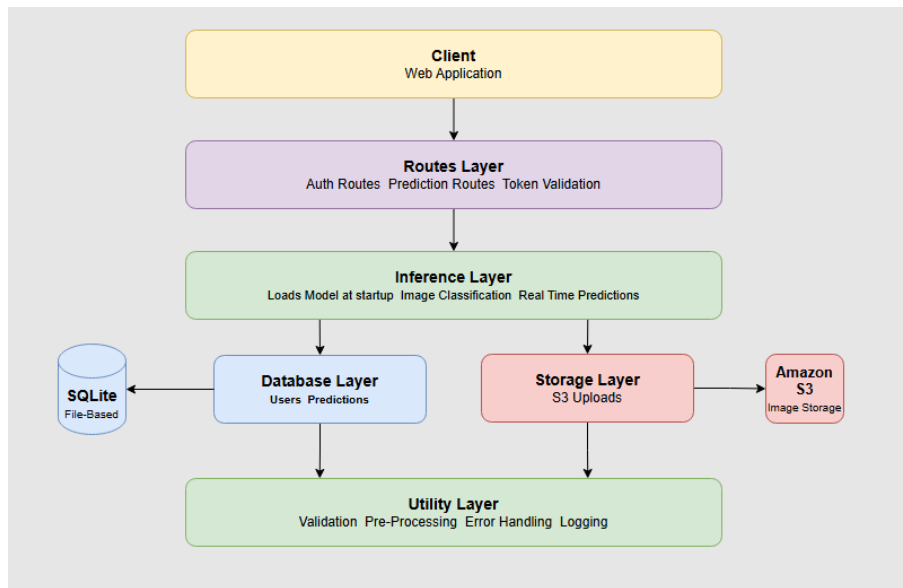


Figure 2.2: System Structure

Routes Layer

The routes layer handles all client-server communications. In this project, routes are separated into two groups: authentication, and prediction-related routes. Authentication routes handle user registration, login and token validation, while prediction routes handle image-uploads, inference, and retrieval of historical predictions per user. Separation of these sections ensures a modular and scalable structure, allowing features to be added without affecting unrelated sections of the system.

Inference Layer

The inference layer is responsible for managing all the models used for image classification. This layer loads the trained model at startup, ensuring it remains in memory and can be reused across all prediction requests. By loading once at the beginning, the back end has better inference speed to support real-time systems.

Database Layer

The database layer handles all persistent data within the back end, including user accounts, authentication details, and prediction history. SQLite is used as the storage engine, providing a lightweight, file based relational database, well suited to the scale and deployment required by this system. This layer also defines the structure of the users and predictions tables. [7]

Storage Layer

The storage layer handles all interactions with Amazon S3, which serves as an external, persistent storage for uploaded images. When a user uploads an image, the back end uploads the image to the designated S3 bucket through this layer. This layer handles the generation of unique object keys, managing S3 upload requests, and ensuring images are securely stored. [8]

Utility Layer

The utility layer handles background functions of the back end components. It deals with common operations such as image validation, pre-processing helpers, error handling and logging. This layer ensures an image meets the required formatting and sizing constraints of the inference layer before being passed on. It also standardises error responses and manages internal logging, allowing the system to handle invalid requests or process failures consistently. This layer reduces duplication within the back end and supports a more maintainable architecture.

2.2.3 API Design

Overview

The API in this project allows the React front end to interact with the system through a FastAPI interface. It exposes endpoints for user authentication and model predictions, acting as a communication layer between the front end and the back end ML and SQLite database. [4]

API Endpoints

Authentication Endpoints

/user/auth/register

Method: POST

Auth: Not required

Status: 200 OK, 401 Email exists

Request (UserCreate):

name	string
email	string
password	string

Response (Token):

access_token	string
token_type	Bearer

/user/auth/login

Method: POST

Auth: Not required

Status: 200 OK, 401 Invalid credentials

Request (UserLogin):

email	string
password	string

Response (Token):

access_token	string
token_type	Bearer

Prediction Endpoints

/api/uploadFile

Method: POST

Auth: Required

Status: 200 OK, 409 Duplicate, 401 Invalid token

Description: Uploads an image, stores it in S3, performs model inference, stores the prediction in SQLite, and returns the prediction object.

Request: file UploadFile

	pred_id	integer
	image_name	string
	prediction	real/fake
Response (PredDB):	confidence	float
	date_time	datetime
	user_id	integer
	user_name	string

/api/predictions

Method: GET

Auth: Required

Status: 200 OK, 401 Invalid token

Description: Returns all predictions associated with the authenticated user.

	pred_id	integer
	image_name	string
	prediction	real/fake
Response (list[PredRead]):	confidence	float
	date_time	datetime
	user_id	integer
	user_name	string

2.2.4 Request/Response Models (Pydantic)

FastAPI uses Pydantic models to validate incoming data and structure outgoing responses. The following models define the schema for all API interactions. [9]

UserCreate

UserCreate		
Fields	name	string
	email	string
	password	string

UserLogin

UserLogin		
Fields	email	string
	password	string

Token

Token		
Fields	access_token	string
	token_type	string

PredCreate

PredCreate		
Fields	image_name	string
	prediction	string
	confidence	string
	date_time	datetime

PredRead

PredRead		
Fields	pred_id	integer
	image_name	string
	prediction	string
	confidence	string
	date_time	datetime
	user_id	integer
	user_name	string

2.2.5 Authentication & Security (JWT)

Token Generation

This system uses JSON Web Token (JWT) based authentication. This ensures that only authenticated users can access protected endpoints such as those related to image uploads and prediction history. JWT based authentication is lightweight and well suited to this type of system due to it not needing server-side session storage.

During registration and login, the back end issues a JWT containing the users unique ID. This token is generated using a secret key and the HS256 signing algorithm. The payload includes the `sub` (subject) field in the `get_current_user` endpoint which stores the users’ ID which allows the back end to identify the user in any request that follows. This token must be included in all protected endpoints within the header: **Authorization: Bearer <token>**

Token Structure

Each JWT consists of three elements:

- **Header** - Specifies the signing algorithm (HS256)
- **Payload** - Contains user ID and metadata
- **Signature** - Ensures token has not been tampered with

The token’s signature is verified every request to ensure authenticity.

Token Validation

Protected routes depend on the `get_current_user` function, which extracts the active token using FastAPI’s OAuth2 bearer scheme. The token is decoded using the same secret key and signing algorithm used to create it. If an invalid token or no token is received, the request is rejected. [10]

Password Hashing

User passwords are not stored in plaintext at any stage. During registration, passwords are hashed using `bcrypt` before being saved to the database. During login the password entered is hashed and compared against the stored hash. This ensures that even if the database was compromised, all passwords would remain protected. [11]

2.2.6 Model Integration

This subsection details how the ML model is integrated into the back end and how inference is performed.

Model Loading

The trained PixelProbe model is loaded once at startup and stored as a global object. This approach removes the need to reload the model with each request allowing low latency inference to be complete. The model contains a custom `LiteMHSA` layer which is registered during loading. Keeping the loaded model in memory ensures that all prediction requests use the same model instance.

Pre-Processing

Before passing an image to the model, it must be resized to mirror the same edits made to images during training. The back end resizes the image to the required 224 X 224 pixels and converts the image to an array suitable for inference. This alignment to the training preprocessing is essential for model consistency and reliability.

Inference

Once pre-processing is complete the image is passed onto the model in its correct format to generate a probability distribution across its two classes: **real** and **fake**. The back end selects the class with the highest probability and extracts its confidence score. These values are then converted into human readable labels and outputted by the model. This workflow ensures that the model outputs consistently in a format usable for display in the frontend.

API Endpoint Integration

The model is integrated into the `/api/uploadFile` endpoint which orchestrates the full prediction pipeline. When an image is uploaded, it is stored temporarily, validated, and forwarded to the preprocessing and inference components. After classification is complete, the resulting prediction and confidence score are combined with timestamp and user ID metadata. This is then returned to the client and stored in the SQLite database, allowing persistent user-specific prediction history. The image itself is then uploaded to an S3 bucket to allow scalable and durable storage.

Error Handling

The integration layer has several safeguards to ensure consistent and reliable operation. Invalid or unreadable images are rejected in pre-processing, and inference errors are shown as HTTP responses. Duplicated prediction attempts trigger conflicts to prevent redundant database entries. These actions ensure model integration behaves correctly in all circumstances, improving the overall robustness of the system.

2.2.7 SQLite

The SQLite database provides a lightweight, persistent storage for all user and prediction related data generated. It is a file-based relational database suited to the scale and requirements of this project, offering low-overhead storage without needing a dedicated database server. [7]

Database Structure

Two primary tables are defined in the back end: **UserDB** and **PredDB**. **UserDB** stores all registered user accounts, which includes identifiers, names, email addresses and hashed passwords. **PredDB** stores all prediction records, including the image name, prediction class, confidence score, timestamp and the identifier and name of the user that uploaded the image. This structure allows the linking of predictions to specific users, allowing for personalised prediction histories.

ORM (Object-Relational Mapping)

The SQLite database is accessed through SQLAlchemy, which uses an ORM layer which extracts raw SQL queries into Python objects. This improves maintainability and readability while ensuring consistent database interaction within the system. SQLAlchemy sessions are dependency-injected into FastAPI endpoints, allowing all requests to operate with its own database context. [12]

Prediction Storage

When an image is uploaded for classification, the results are packaged into a **PredDB** object and committed to the SQLite database. Each entry includes the prediction output and associated user metadata, allowing the system to maintain a complete record of user activity. This information is later retrieved in the `/api/predictions` endpoint, which returns all prediction linked to the current authenticated user. This workflow allows the frontend to display a persistent, user specific prediction history.

User Authentication

During registration, new users are inserted into the `UserDB` table with bcrypt hashed passwords. During Login, the back end needs SQLite to confirm credentials and retrieve the users unique ID, which is embedded into the JWT token. This ensures authentication is secure and closely linked to the database layer.

Error Handling

Duplicate entries, such as repeated predictions for the same image trigger integrity errors that are shown as conflicts to the user. SQLAlchemy’s transaction management ensures that failed operations do not corrupt the database state. This ensures a reliable storage layer, support overall stability of the system.

2.3 Frontend - React

The frontend servers as the middle layer between the user and the PixelProbe system. Its role is to provide an responsive, easy to use and secure interface, allowing the user to upload images, Login or Register to the system, and view their prediction history. While the back end handles model inference, authentication logic and data storage, the frontend is responsible for handling user experience and ensuring all interactions with the back end are completed smoothly and securely.

2.3.1 Technology Stack

ReactJS

React is a modern JavaScript library designed for building interactive user interfaces through a component-based architecture. Instead of manipulating the browser DOM directly, React maintains a virtual DOM where all UI updates are computed. Once changes are confirmed, React applies only the necessary updates to the real DOM, reducing rendering overhead and improving performance. [13]

For this project, React provides a structured approach to building the PixelProbe frontend. Its component-based model allows the interface to be divided into sections such as authentication forms, image upload components, and prediction history displays. This modular approach aligns with the overall system architecture and aids in the maintainability of the application as it grows.

React handles user input, authentication state, and back end response through hooks. In this project the frontend communicates with the FastAPI back end using the fetch API, attaching a stored JWT token to all protected requests through the `Authorization: Bearer <token>` header. Images are sent using `multipart/form-data`, and authentication requests use JSON-encoded bodies. [14] [15] [16]

JSX

JSX is a syntax used in React to define component structure using a HTML-style markup embedded directly into JavaScript. This allows the UI layout and logic to be expressed together in a clear readable form. JSX is compiled into standard JavaScript function calls, allowing React to update the virtual DOM efficiently [17]

In this project, JSX is used to structure interface components such as the authentication forms, the image upload, and historical prediction display. It supports a modular, maintainable frontend aligned with the overall design of PixelProbe’s system.

JavaScript

JavaScript provides the core logic of the PixelProbe frontend, allowing dynamic behaviour inside React components while supporting user-driven actions. It handles event processing, state transitions, rendering, and asynchronous operations required to communicate with the FastAPI back end. JavaScript allows the interface to remain responsive while performing tasks such as image uploads, authentication and retrieving historical predictions [18]

In this project, JavaScript is used to create functional components, manage state through hooks and implement logic relating to user actions. It facilitates API communication through the `fetch` API, allowing the frontend to send JSON-encoded authentication data, attach JWT to protected requests, and send images using `multipart/form-data`. This makes JavaScript a suitable foundation for building a responsive frontend.

CSS

CSS is used to define the visual layout, styling, and presentation of the UI. It controls the structure and spacing of components, the colour scheme, and the behaviour of the application on different screen sizes. By separating styling from component logic, CSS allows for a clear visual hierarchy and ensures the interface is accessible and consistent for all users.

In this project, CSS is applied to style interface elements such as the navigation bar, authentication forms, the image upload view and the prediction history display. It allows the frontend to have a clean design while keeping dynamic content such as prediction results and uploaded images organised. This contributes to a user friendly interface aligning with the overall architecture. [19]

Node.js and NPM

Node.js provides the development environment for the frontend, allowing the application to be built, tested, and served locally. It has a lightweight JavaScript runtime supporting modern tools, fast build processes and efficient dependency management. NPM (Node Package Manager), is used to install external libraries required by the React application, including React itself.

In this project, Node.js hosts the local server. NPM manages project dependencies and ensures consistency across build environments. Together they form the foundations of the frontend development workflow, allowing for modular and maintainable code that integrates easily with the back end. [20]

2.3.2 Component Structure

The PixelProbe front end follows a modular structure which allows all components to operate independently of each other. This follows React’s standard modular based style. Each component is responsible for a specific task, all combined to create one complete system. This modular structure aids both maintainability and scalability of the project.

The UserLogin component handles all authentication related functions. They intake user information, validate form data, and forward credentials to the back end through JSON requests. These handle the storage and retrieval of the JWT token, allowing secure access to protected routes and ensuring the authentication state persists through page reloads.

The fileUpload and predictButton components are joined together to implement core functions of the system. The fileUpload component allows users to upload an image for prediction, completes client-side user authentication and sends the file to the back end. When a response

is received the result component displays the prediction, confidence score and a gauge showing the percentage confidence in a clear format.

The `imageGallery` component displays the historical predictions of the currently active user. It renders a list containing image previews pulled from S3, alongside their name, prediction, confidence and time stamp. This gives the user a persistent view of their previous uploads to the system.

The `NavBar` component provides easy access to each page. This allows the user to navigate across components such as the `ImageUpload` page, the history predictions and login/homepage with ease.

2.3.3 State Management

State management is handled through React’s built in hooks, which track user input, authentication status, and back end responses. The `useState` hook stores the local component’s state, allowing the UI to update as users interact with different components. This ensures the UI is always synchronised with information coming from back end responses without the need for manual updates.

For persistent data such as the users JWT token and prediction history pulled from the back end uses a mix of React context and browser storage. The token is stored in the `localStorage`, allowing the frontend to attach it to protected API requests across page reloads. This ensures a secure and consistent authentication process aligned with the back end.

2.3.4 API Communication

The frontend communicates with the FastAPI through a dedicated `fetch` API. All requests are created based on the base URL of the back end, this ensures paths are consistent. [15]

Authentication uses JSON POST requests to relay registration and login data. Once login is successful, the back end returns a JWT token, unique to the user and it is stored in the frontend’s `localStorage`. This token is then attached to all protected requests through the `Authorization: Bearer <token>` header, allowing secure access to personalised prediction and history endpoints. This ensures that the same authorization is used across both the frontend and back end.

Prediction requests use `multipart/form-data` to send binary image files to the back end. The frontend creates a `FormData` object containing the uploaded file and sends it to the back end’s prediction endpoint. Once the back end returns the prediction and confidence score, the response is displayed through the `fileUpload` component. All API functions are completed using

asynchronous operations, allowing the interface to continue working during communication with the back end. Each request checks the `response.ok` flag to check for back end errors and display them when necessary. This allows for consistent error handling and simple communication between the React frontend and FastAPI back end.

2.3.5 Routing and Navigation

Routing in the PixelProbe frontend is done through React Router. React router is a routing library allowing easy navigation between different views without needing to reload the page. The routing is all defined within the `App.js` file, which serves as the main page/controller for all pages in the application. [21]

Routes Structure

- `"/`: The homepage, which shows the PixelProbe logo alongside a login/registration form.
- `"/predictions"`: The image upload page, allows logged in users to upload images for prediction.
- `"/history"`: The prediction history page, displays all previous predictions associated with the current user.

Navigation Bar Integration

The `Navbar` component is outside of the main block, this ensures it is visible across all pages. This allows the user to have quick access to the homepage, image upload and history pages.

Error Handling

While the back end enforces authentication for certain endpoints, the frontend does not prevent users from accessing the `/predictions` or `/history` routes. Users can navigate between all pages without a valid JWT token. Any api requests made from this pages will be rejected if there is no valid user logged in.

2.3.6 Security

Frontend security is crucial to ensure all user data, authentication tokens and API interactions are protected. The back end enforces access control through JWT validation, while the frontend implements certain measures to keep secure communication and prevent unauthorised access.

JWT Storage and Handling

The frontend stores the JWT returned at login within the browser's `localStorage`. This allows the authentication to persist across page reloads and allows the token to be attached to all protected requests through the `Authorization: Bearer <token>` header. `localStorage` can be vulnerable to token theft, however it provides a simple method for maintaining session state in single page applications.

Protected API Requests

All requests to protected endpoints, such as ImageUploads and prediction history, must include the JWT token in the request header. If there is no token, it is expired or tampered with, the back end rejects the request. The frontend checks these responses and can prompt the user to re-authenticate if necessary. This ensures only verified users can access sensitive data.

2.4 Database - SQLite

SQLite is the main data storage container for this project. It provides a lightweight, file based relational database that suits the scale and requirements of this system. SQLite's low overhead storage without the need for a dedicated server is ideal for this application where simplicity and reliability is most important.

SQLite operates as an embedded library within FastAPI. This reduces complexity in development, and ensures all data is stored persistently in a single `.db` file on the server. This supports fast development, easy deployment on AWS and predictable performance.

2.4.1 Schema

The database consists of two primary tables: **UserDB** and **PredDB**. These tables store all user and prediction related data.

Database Tables

UserDB

UserDB

The **UserDB** table stores all registered users and their associated account data.

Fields	<code>user_id</code>	Primary key
	<code>name</code>	Display name for prediction history
	<code>email</code>	Unique, prevents duplicate accounts
	<code>password_hash</code>	bcrypt-hashed password
	<code>is_active</code>	Boolean flag for account status

Notes: `user_id` links users to prediction records. Passwords are always hashed before storage. [11]

PredDB

PredDB

The **PredDB** table stores all prediction entries generated by the model.

	<code>pred_id</code>	Primary key
	<code>image_name</code>	Name of uploaded image (stored in S3)
	<code>prediction</code>	Model output (real/fake)
Fields	<code>confidence</code>	Confidence score (string)
	<code>date_time</code>	Timestamp of prediction
	<code>user_id</code>	Foreign key to UserDB . <code>user_id</code>
	<code>user_name</code>	Cached username for fast retrieval

The foreign keys form a relationship between predictions and users to ensure each prediction is tied to a specific user, and can be used to retrieve personalised prediction histories.

2.4.2 ORM Integration (SQLAlchemy)

SQLAlchemy is used as the Object-Relational Mapper (ORM) for all database interactions. The ORM provides a way to define tables, query the database and manage transactions without writing raw SQL. This improves readability and ensures consistent interactions with the database.

Through SQLAlchemy’s syntax, each table is created as a Python Class. These classes define the schema, data types and constraints, allowing SQLite to generate corresponding SQLite tables at startup. This means schemas do not need to be manually created and database structure always matches the back end models.

FastAPI integrates with SQLite through dependency-injected database sessions. Each request has its own instance, preventing conflicts between parallel operations. Sessions handle commits, rollbacks and connection management automatically. This allows the back end to perform create, read and update operations safely.

The ORM layer offers clean abstraction over raw SQL, supports schema generation and ensures reliable database access through the system.

2.4.3 Database Operations

The database layer handles all interactions with the SQLite file, which includes inserting new records, retrieving stored data, and managing errors that occur during operation. SQLAlchemy ensures each request operates within its own instance, allowing safe, consistent access.

Create Operations

Create operations happen when new users are registered or when a new prediction is generated. A new SQLAlchemy model instance is created, and added to the active session, and committed to the database.

Read Operations

Read operations are used through the back end to access stored information. Examples include fetching users in login, checking for existing predictions, or returning prediction history linked to an authenticated user. SQLAlchemy’s query interface abstracts raw SQL into Python, allowing easy filtering, sorting and lookup operations while keeping a clean codebase.

Error Handling

SQLAlchemy provides built-in mechanisms to detect and manage database errors. Integrity issues- such as duplicate emails or repeat predictions trigger exceptions which are then converted to HTTP responses. If operations fail the session is rolled back automatically, ensuring a clean database free of corrupted data or partial writes.

2.4.4 Data persistence and Storage

SQLite provides persistent, file-based storage for all user and prediction data. All information is stored in a single `.db` file on the server, allowing the back end to retain state across restarts. This design ensures consistent and predictable performance on a single-instance AWS setup. SQLite writes directly to the disk; all committed transactions, such as new user registrations and prediction entries are permanently stored unless removed. SQLAlchemy manages these writes through its sessions, ensuring every operation is committed safely and rolled back when errors happen.

The file-based style of SQLite makes backups and migrations straightforward. The entire database can be copied, archived or transferred in a single file, allowing for easy maintenance and future upgrades to more scalable engines.

2.4.5 Security

Security within the database layer focuses on protecting user credentials, maintaining data integrity, and ensuring only authenticated users can access sensitive information. The back end implements several measures to ensure stored data remains secure and consistent.

User passwords are never stored in plaintext. The back end hashes the password using `bcrypt` before inserting. During login, a plaintext password is entered and hashed, then compared with the stored hashed password. This ensures that if the database file is directly accessed, no usable credentials are exposed.

Access to prediction records is protected through JWT-based authentication. Each request to protected endpoints include a valid token, allowing the back end to identify the user and only output their own prediction history. This prevents unauthorized users from viewing or modifying other user’s data.

Together, these measures provide a secure foundation for storing user information and prediction data.

2.4.6 Future Improvements

SQLite is suited to the scale of this project, several improvements could be made as the system expands.

First improvement as the scale of the project grows would be to upgrade to a more scalable relational database such as PostgreSQL or MySQL. These engines provide better concurrency support, improved indexing, and better handling of high-volume workloads. This would allow the system to support more users, parallel requests and larger prediction histories without effecting performance.

Integrating soft deletes or archive tables would allow for old prediction records to be moved out of the main tables without being permanently deleted. This would reduce clutter in frequently accessed tables, and enable compliance with data retention policies.

Introducing a migration tool such as Alembic would allow the database schema to evolve over time. This ensures that structure changes, such as adding new fields or changing constraints, can be applied across environments throughout development.

2.5 Cloud Architecture - S3

2.5.1 Cloud Overview

A Cloud infrastructure is used in this project to provide a scalable and available storage for user uploaded images. Cloud services are well suited to this system as they allow storage and compute resources to grow independently as demand increases

AWS is used as the cloud provider, offering reliable object storage through S3 and a stable host environment for the back end. AWS ensures uploaded images are stored securely, persistently and can be accessed by the back end at anytime without managing local storage or server-side file systems.

2.5.2 S3 Storage

S3 acts as the primary storage location for all uploaded images. S3 provides large storage capacity and simple integration with Python through the AWS SDK.

Bucket structure

A single S3 bucket is used to store uploaded images. Each image is uploaded using its original filename, and the back end prevents duplicate prediction entries. The bucket structure is flat, storing each object at root level under its filename.

Image Upload Workflow

The uploaded file is temporarily saved in the uploads folder, the back end uses the boto3 client to upload the file to S3, after a successful upload, the image is passed to the Tensorflow model for inference. The result is then stored in SQLite, including the S3 object key (`image_name`). The temporary local file is then deleted.

Access Control and Security

The back end authenticates with S3 through AWS credentials and a session token. These credentials only allow the required operations (upload and read). All communication with S3 uses HTTPS, ensuring images are encrypted in transit. Access to the bucket is restricted to the back end only; users never interact direct with S3.

2.5.3 Back end Hosting Environment

Deployment Architecture

The FastAPI back end is hosted on an AWS instance. At startup, the back end loads the TensorFlow model and initialises the S3 client using the configured AWS credentials. The back end exposes the API’s endpoints for authentication, image upload, and prediction results, and communicates with SQLite and S3 during each request.

Integration between Back end and S3

Integration with S3 is done through the `boto3` client created at startup. The back end uses this to upload images directly to the S3 bucket. The filename is stored in the `PredDB` table, allowing the frontend to reference the correct image when displaying prediction history. This keeps storage, inference and database layers separated.

2.5.4 Security

Security is implemented at multiple levels. AWS credentials are required to access the S3 bucket, IAM permissions restrict the back end to only necessary S3 operations, JWT authentication protects user and prediction endpoints, temporary files are deleted immediately after upload and no sensitive data is stored in the frontend or client code.

2.5.5 Scalability and Future Improvements

The cloud architecture is designed to scale as the system grows. S3 automatically handles increased storage volume without configuration changes. The back end could be scaled by integrating a multi-instance setup rather than one EC2 instance.

3 Machine Learning

3.1 Datasets

PixelProbe models are trained using publicly available datasets stored in the project route, over the course of this project four datasets have been used. Each dataset contains two top-level splits: `train` and `test`. Each split is separated into two classes that represent real and AI-Generated images. The datasets are loaded using Tensorflow’s `image_dataset_from_directory` function which automatically assigns integer labels and resizes all images to the 224 x 224 pixels required by the EfficientNet model.

A batch size of 64 is used for training and testing. The loader shuffles training data and ensures consistent class ordering through `class_names`. This allows the model to learn from a balanced set of real and AI-Generated images while maintaining a clean separation between training and evaluation data.

3.2 Model Architecture

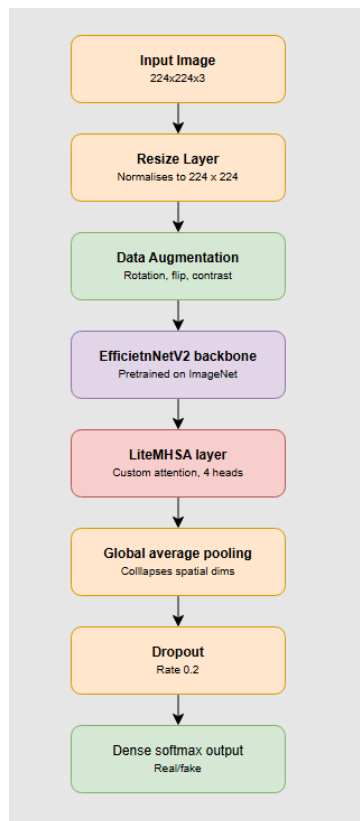


Figure 3.1: Model Architecture

PixelProbe is built on top of the EfficientNet family architecture, a group of modern CNNs pre-trained on ImageNet. The backbone is loaded without the classification head, allowing the model to be adapted to the classification task.

To enhance feature extraction a custom Lite Multi-Head Self-Attention (LiteMHSA) layer has been added after EfficientNet’s feature maps. This layer performs lightweight attention across spatial dimensions, improving the model’s ability to capture fine-grained artifacts commonly found.

The combination of pre-trained convolutional features and lightweight attention provides a good balance between accuracy and efficiency.

3.3 Training Process

The training process in this project is split into two phases. This stabilises learning and preserves pre-trained features.

3.3.1 Phase 1 - Train Classification Head only

The EfficientNet backbone is frozen, and only the new layers (LiteMHSA, pooling, dropout, dense output) are trained.

The model is compiled with:

- **Optimizer:** Adam
- **Loss:** Sparse categorical cross-entropy
- **Metrics:** Accuracy

Training runs for up to 15 epochs with early stopping and model check-pointing enabled.

3.3.2 Phase 2 - Fine-Tuning

Once the head stabilises, the top 20 layers are unfrozen. Batch normalisation layers remain frozen to avoid destabilising pre-trained weights.

Fine-tuning uses:

- **Optimizer:** Adam with reduced learning rate
- **Loss:** Sparse categorical cross-entropy

This phase runs up to 30 epochs, again with early stopping and check-pointing.

3.3.3 Callbacks

The training process utilises:

- **Early Stopping:** Stops training when validation loss stops improving
- **Model-checkpoint:** saves best model to `BE/model/best_model.keras`
- **Tensor-board:** logs training metrics for visualisation

After training, the model and combined training history is saved for future plotting.

3.4 Evaluation

3.4.1 Accuracy

Test accuracy is computed using scikit-learn’s `accuracy_score` function. This provides a clear measure of how well the model generalises to unseen images.

3.4.2 Confusion Matrix

A confusion matrix is generated to evaluate performance per class. This is done using sklearn’s `confusion_matrix` and `ConfusionMatrixDisplay` functions.

The confusion matrix highlights how often a model gets confused between real images and AI-Generated ones. This helps to understand failure cases and identify possible dataset biases.

3.4.3 Output and Storage

All plots are saved in a local plots folder. The script prints the final test accuracy and confirms all plots have been generated successfully.

This provides a complete overview of model performance, combining metrics with visual plots to assess the reliability of the model

3.5 Results

The performance of PixelProbe’s models was evaluated across multiple dimensions: training behaviour, test accuracy, confusion matrix analysis and cross-dataset generalisation.

3.5.1 Training Behaviour

Multiple models were trained using different datasets and model configurations. The training curves consistently showed expected behaviour of a convolutional model fine-tuned.

The following trends were observed across all model versions:

- Training loss decreased steadily which indicated correct optimisation
- Validation loss fluctuated in early epochs but stabilises as the model progresses
- Training accuracy consistently increased, showing effective learning
- Validation accuracy remained close to training accuracy indicating low over-fitting

These behaviours were consistent across shorter epoch runs of around 5 epochs up to extended runs ranging from 20-25 epochs. The consistency of these results indicated the training pipeline generalises well to different data distributions.

3.5.2 Test Set Performance

Each dataset contained a held out test set for performance evaluation. The evaluation script computed predictions based on these sets using the final model from training. The results showed strong performance, with high accuracy and balanced classifications across both classes.

These trends were observed across confusion matrices:

- High true positive counts on both real and fake classes
- Low misclassification rates indicated strong classification ability
- Symmetrical performance indicated no notable bias towards either of the classes

Across all model versions, the generated matrices showed similar patterns. Most samples in the respective test sets fall along the diagonal, which confirms reliable classification.

3.5.3 Cross-Dataset Generalisation

To assess the robustness outside of the training sets, multiple PixelProbe versions were evaluated on external datasets. These included:

- ZHANG [22]
- CIFAKE [23]
- GENImage [24]
- Places365 [25]
- GHAI [26]

The results of five models trained on different datasets were compared. The key findings were:

- Models trained on the ZHANG dataset performed well on its own set and the GHAI set but were less effective on the CIFAKE dataset.
- Models trained on the CIFAKE dataset performed well on its own set but struggled on other datasets.
- Models trained on GenImage and the joint GenImage/Places365 datasets achieved the best performance across all dataset.

3.5.4 Overall Assessment

The results from training curves, confusion matrices and cross dataset testing shows that the PixelProbe architecture is:

- Stable in training
- Accurate on test data from own set
- Robust across multiple datasets
- Effective at detecting generative artifacts

The integration of a LiteMHSA attention layer enhances EfficientNet’s sensitivity to subtle inconsistencies typical in AI-Generated media. The two-phase training strategy further improved generalisation by preserving pre-trained features while also allowing targeted fine-tuning.

Overall, the results confirm that PixelProbe is a reliable and effective tool for detecting AI-Generated images.

4 Implementation & Results

4.1 Back end

4.1.1 Structure

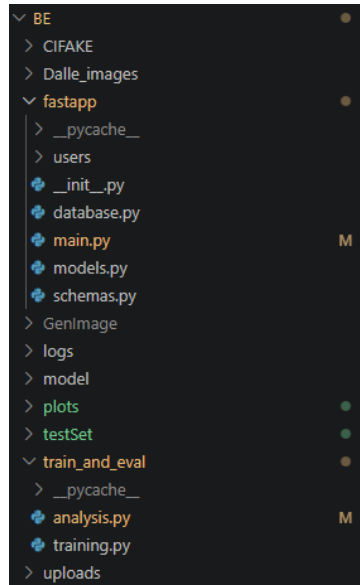


Figure 4.1: Back end Folder Structure

The project directory is organised into several key components, each supporting a specific aspect of the system’s functionality.

The `CIFAKE`, `Dalle_images`, `GenImage` and `testSet` directories contain the datasets used throughout model, development. These include the training datasets (CIFAKE, DALL_E images, and GenImage) as well as the test set used for final evaluation of model performance.

The `fastapp` directory holds all back end code relating to the FastAPI application. This includes `main.py`, where the API endpoints are defined, `database.py`, where the SQLite engine and session are set, `models.py` where ORM models are defined, and `schemas.py`, which holds the Pydantic schemas for data validation.

The `Uploads` directory temporarily holds user-submitted images during inference. Before S3 integration, this directory was used as local storage for uploaded files.

The `model` directory holds all trained models in `.keras` files alongside their `.json` training history logs. These are later used by `analysis.py` for evaluation and by the FastAPI back end during inference.

4.1.2 FastAPI

PixelProbe’s back end is implemented through FastAPI. This section outlines the implementation of the API layer, including application setup, middleware configuration, routing, dependency management, model integration, database integration and S3 storage.

Application Setup

The FastAPI application is initialised in `fastapp/main.py`. The core app instance is created and configured with CORS middleware to allow the React frontend to communicate with the back end:

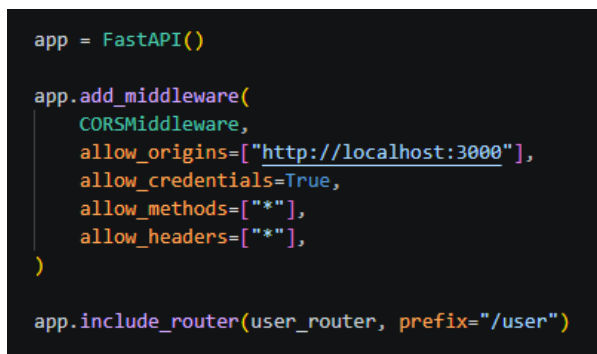
```
app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Authentication routes are defined in a separate script and mounted under the `/user` prefix:

```
app.include_router(user_router, prefix="/user")
```

The `main.py` file, including imports, router registration, and global configuration is shown in Figure 4.2.



```
app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(user_router, prefix="/user")
```

Figure 4.2: `main.py` showing FastAPI app setup, middleware, and router registration.

Database (SQLite & SQLAlchemy)

The back end uses SQLAlchemy to communicate with the SQLite database. The database schema is created at startup using the ORM models defined in `models.py`

```
Base.metadata.create_all(bind=engine)
```

A session is provided to each request using a FastAPI dependency

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

The ORM models `UserDB` and `PredDB` outline the structure of the users and predictions tables. A screenshot of the model definitions is shown in Figure 4.3.

```

class PredDB(Base):
    __tablename__ = "predictions"

    pred_id: Mapped[int] = mapped_column(primary_key=True, index=True)
    image_name: Mapped[str] = mapped_column(String, nullable=False)
    prediction: Mapped[str] = mapped_column(String, nullable=False)
    confidence: Mapped[str] = mapped_column(String, nullable=False)
    date_time: Mapped[datetime] = mapped_column(DateTime, default=datetime.now, nullable=False)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.user_id"), nullable=False)
    user_name: Mapped[str] = mapped_column(String, nullable=False)

class UserDB(Base):
    __tablename__ = "users"

    user_id : Mapped[int] = mapped_column(primary_key=True, index=True)
    name: Mapped[str] = mapped_column(String, nullable=False)
    email : Mapped[str] = mapped_column(String, nullable=False)
    hashed_password : Mapped[str] = mapped_column(String, nullable=False)
    is_active : Mapped[bool] = mapped_column(Boolean, nullable=False)

```

Figure 4.3: SQLAlchemy ORM models for users and predictions in `models.py`.

Model Integration

The trained model is loaded once at startup and stored as a global object. Because a LiteMHSA layer is included, it must be registered through the `custom_objects` argument:

```

model = tf.keras.models.load_model(
    "BE/model/best_model.keras",
    custom_objects={"LiteMHSA": LiteMHSA}
)
class_labels = ["real", "fake"]

```

The `classify_image` function completes preprocessing and passes the image to the model for inference:

```

def classify_image(image_path):
    img = image.load_img(image_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)

    preds = model.predict(x)

    predicted_class = class_labels[np.argmax(preds[0])]
    confidence = float(preds[0][np.argmax(preds[0])])
    return predicted_class, confidence

```

A screenshot of the full inference helper implementation, including debug logs, is shown in Figure 4.4.

```
def classify_image(image_path):
    img = image.load_img(image_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)

    preds = model.predict(x)

    print(f"Raw predictions: {preds}")
    print(f"Prediction shape: {preds.shape}")
    print(f"Max value: {np.max(preds[0])}")
    print(f"Argmax: {np.argmax(preds[0])}")

    predicted_class = class_labels[np.argmax(preds[0])]
    confidence = float(preds[0][np.argmax(preds[0])])
    return predicted_class, confidence
```

Figure 4.4: `classify_image` helper used for model inference.

S3 Storage

Uploaded images are stored in an Amazon S3 bucket. A boto3 client is configured using AWS credentials and region information.

```
s3 = boto3.client(
    "s3",
    aws_access_key_id=AWS_ACCESS_KEY,
    aws_secret_access_key=AWS_SECRET_KEY,
    aws_session_token=AWS_SESSION_TOKEN,
    region_name=AWS_REGION
)
```

Images are uploaded to the bucket using:

```
s3.upload_file(file_path, BUCKET_NAME, file.filename)
```

The full S3 configuration in `main.py` is shown in Figure 4.5.

```
s3 = boto3.client(
    "s3",
    aws_access_key_id=AWS_ACCESS_KEY,
    aws_secret_access_key=AWS_SECRET_KEY,
    aws_session_token=AWS_SESSION_TOKEN,
    region_name=AWS_REGION
)
```

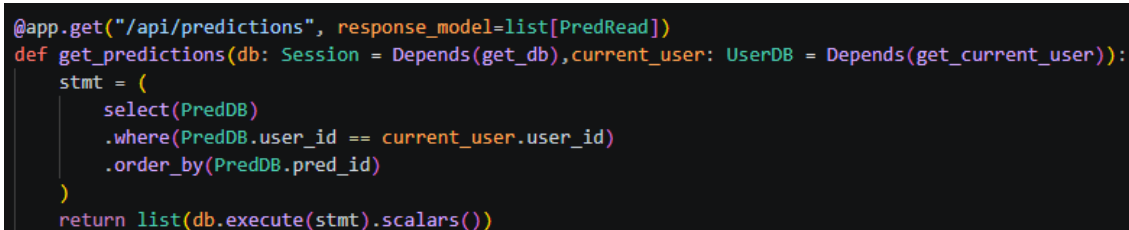
Figure 4.5: S3 client configuration in the FastAPI back end.

Prediction History

The `/api/predictions` endpoint returns all predictions associated with the current user. It uses the `get_current_user` dependency to identify the user from the JWT token and SQLAlchemy to query the database:

```
@app.get("/api/predictions", response_model=list[PredRead])
def get_predictions(
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user)
):
    stmt = (
        select(PredDB)
        .where(PredDB.user_id == current_user.user_id)
        .order_by(PredDB.pred_id)
    )
    return list(db.execute(stmt).scalars())
```

A screenshot of the full predictions route is shown in Figure 4.6.



```
@app.get("/api/predictions", response_model=list[PredRead])
def get_predictions(db: Session = Depends(get_db), current_user: UserDB = Depends(get_current_user)):
    stmt = (
        select(PredDB)
        .where(PredDB.user_id == current_user.user_id)
        .order_by(PredDB.pred_id)
    )
    return list(db.execute(stmt).scalars())
```

Figure 4.6: Implementation of the `/api/predictions` endpoint in `main.py`.

Image Upload and Inference

The core functionality of the back end is implemented through the `/api/uploadfile` endpoint. This endpoint:

- Receives an uploaded image from the frontend
- Saves it temporarily to the `uploads` directory
- Uploads the file to S3
- Completes inference using `classify_image`
- Stores the prediction in the SQLite database
- Returns the prediction object to the client

The core logic is as below:

```
@app.post("/api/uploadfile")
async def create_upload_file(file: UploadFile = File(...), db: Session = Depends(get_db)):

    existing = db.query(PredDB).filter(
        PredDB.image_name == file.filename,
        PredDB.user_id == current_user.user_id
    ).first()

    if existing:
        raise HTTPException(
            status_code=409,
            detail=f"Image '{file.filename}' has already been processed."
        )

    file_path = f"BE/uploads/{file.filename}"

    try:
        with open(file_path, "wb") as f:
            f.write(file.file.read())

        s3.upload_file(file_path, BUCKET_NAME, file.filename)
        pred_class, conf = classify_image(file_path)

        prediction = PredDB(
            image_name=file.filename,
            prediction=pred_class,
            confidence=conf,
            date_time=datetime.now(),
            user_id=current_user.user_id,
            user_name=current_user.name
        )

        db.add(prediction)
        db.commit()
        db.refresh(prediction)

    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail=f"Image '{file.filename}' has already been processed.")

    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Upload failed: {str(e)}")

    finally:
        if os.path.exists(file_path):
            os.remove(file_path)

    return prediction
```

Integrity errors, such as duplicate predictions are handles using SQLAlchemy's exception types and mapped to HTTP responses. The full endpoint implementation, including error handling, is shown in Figure 4.7.

```
@app.post("/api/uploadfile")
async def create_upload_file(file: UploadFile = File(...), db: Session = Depends(get_db), current_user: UserDB = Depends(get_current_user)):
    existing = db.query(PredDB).filter(
        PredDB.image_name == file.filename,
        PredDB.user_id == current_user.user_id
    ).first()
    if existing:
        raise HTTPException(
            status_code=409,
            detail=f"Image '{file.filename}' has already been processed."
        )
    file_path = f"BE/uploads/{file.filename}"
    try:
        with open(file_path, "wb") as f:
            f.write(file.file.read())
        s3.upload_file(file_path, BUCKET_NAME, file.filename)
        pred_class, conf = classify_image(file_path)
        prediction = PredDB(
            image_name=file.filename,
            prediction=pred_class,
            confidence=conf,
            date_time=datetime.now(),
            user_id=current_user.user_id,
            user_name=current_user.name
        )
        db.add(prediction)
        db.commit()
        db.refresh(prediction)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail=f"Image '{file.filename}' has already been processed.")
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Upload failed: {str(e)}")
    finally:
        if os.path.exists(file_path):
            os.remove(file_path)
    return prediction
```

Figure 4.7: Full implementation of the /api/uploadfile endpoint in main.py.

Authentication & Security

All protected endpoints depend on the `get_current_user` function, which ensures a valid JWT token is provided in the `Authorization` header. If the token is missing or invalid, the request is rejected. This function is defined in the `dependencies.py` script, which acts as a dependency layer for authentication. It extracts the token using the FastAPI OAuth2 bearer scheme, decodes it using the secret key and algorithm used in creation, and retrieved the related user from the database.

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid authentication",
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception
    user = db.query(UserDB).filter(UserDB.user_id == int(user_id)).first()
    if user is None:
        raise credentials_exception
    return user
```

Figure 4.8: JWT-based authentication and `get_current_user` implementation in the `users` module.

This combination of FastAPI dependencies, JWT authentication, and database user records ensures that only authenticated users can upload images and access their prediction history.

4.1.3 Security

PixelProbe handles authentication through JSON Web Tokens (JWT), bcrypt password hashing, and FastAPI dependency injection. This ensures only authorised users can access protected endpoints such as image upload and retrieving prediction history. Authentication is implemented in the `users` module, which contains registration and login endpoints.

Password Hashing

Passwords are never stored in plaintext. The back end uses the `passlib` library with the bcrypt hashing scheme:

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def get_password_hash(password: str):
    password = password[:72]
    return pwd_context.hash(password)
```

Passwords are truncated into 72 characters as bcrypt ignores characters beyond this limit; enforcing this prevents inconsistent hashing.

Passwords are verified using:

```
def verify_password(plain_password: str, hashed_password: str):
    plain_password = plain_password[:72]
    return pwd_context.verify(plain_password, hashed_password)
```

This ensures that even if the database was accessed, no user credentials can be accessed.

JWT Token Generation

Authentication is implemented through JSON Web Tokens. When a user is registered or logged in, the back end issues a signed token containing the user’s ID and expiration timestamp.

```
SECRET_KEY = "super-secret"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

def create_access_token(user_id: int):
    expire = datetime.now() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {"sub": str(user_id), "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

The token is signed using HS256, ensuring any later modification invalidates the signature.

Registration Endpoint

The `/auth/register` endpoint creates a new user. It check to ensure the inputted email is not already in use, hashes the password, stored the user in the database and assigns a JWT token:

```
@user_router.post("/register", response_model=Token)
def register(user: UserCreate, db: Session = Depends(get_db)):
    existing_user = db.query(UserDB).filter(UserDB.email == user.email).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="Email already registered")

    new_user = UserDB(
        name=user.name,
        email=user.email,
        hashed_password=get_password_hash(user.password),
        is_active=True,
    )

    db.add(new_user)
    db.commit()
    db.refresh(new_user)

    token = create_access_token(new_user.user_id)
    return {"access_token": token, "token_type": "bearer"}
```

This allows users to be authenticated automatically when registered.

Login Endpoint

The `/auth/login` endpoint verifies users credentials and assigns a new JWT token when authenticated:

```
@user_router.post("/login", response_model=Token)
def login(user: UserLogin, db: Session = Depends(get_db)):
    db_user = db.query(UserDB).filter(UserDB.email == user.email).first()

    if not db_user or not verify_password(user.password, db_user.hashed_password):
        raise HTTPException(status_code=400, detail="Invalid credentials")

    token = create_access_token(db_user.user_id)
    return {"access_token": token, "token_type": "bearer"}
```

Invalid credentials returns a 400 Bad request

Token validation and Access Control

Protected endpoint all depend on the `get_current_user` function, which:

- Extracts the token from the Authorization: Bearer <token> header
- Decodes and verifies the JWT signature
- Checks the token expiration timestamp
- retrieves the corresponding user from the database

If any of these steps fail, FastAPI returns a 401 **Unauthorised** response. This ensures only authenticated users can upload images or view prediction history.

Together, these mechanisms provide a secure authentication system that protects user credentials, ensures no unauthorised access and safe interaction with all protected endpoints.

Uploads Directory

The uploads directory acts as a temporary storage for images received from the frontend during prediction. When an image is submitted through React, the back end first writes the raw file here before processing. This temporary storage step is needed because the TensorFlow inference pipeline and the S3 upload client both operate on file paths.

Once the file is saved, the back end performs three tasks:

- The image is uploaded to the S3 bucket for long term storage.
- The local file is passed to the `classify_image` helper for preprocessing before inference
- After inference is complete and the prediction is stored, the temporary file is deleted.

This ensures that the uploads directory does not accumulate unnecessary files and remains lightweight during operation. All permanent storage is handled exclusively through S3, reducing disk usage of the system and improving scalability.

4.2 Frontend

4.2.1 Structure

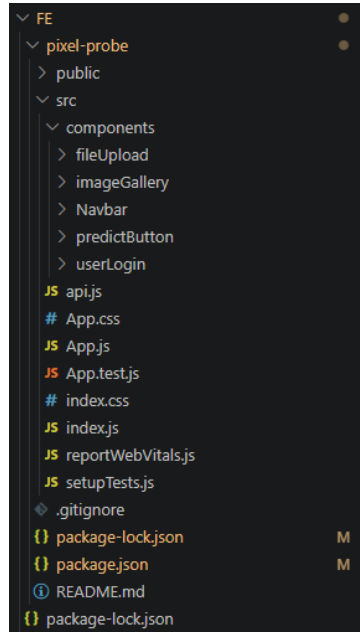


Figure 4.9: Frontend File Structure

PixelProbe’s frontend is organised into several key components, each handling a specific aspect of the systems user-facing functionality.

The **public** directory contains any static assets and the base **index.html** file into which the React application is injected. This file acts as a root container for all components, and any other assets in this directory are global resources such as icons.

The **src** directory deals with all functional logic for the application. This includes **index.js** which mounts the application to the DOM, and **App.js** which defines the application’s layout and routing. Global styling is handled through **App.css** and **index.css**, which ensures a consistent format across all components.

The **components** directory contains all modular elements used in the frontend. These are organised into individual subdirectories based on their function. The **userLogin** directory holds login/registration forms, handling user authentication and token storage.

The **fileUpload** directory holds the image upload interface, which includes file uploads and communication with the back end inference endpoint. The **imageGallery** directory related to rendering the user’s personalised prediction history, including S3 upload image previews, prediction labels, confidence score and timestamps.

The **NavBar** directory handles the navigation bar, which provides easy access between pages. The **predictButton** provides a reusable button used alongside the **fileUpload** component in the inference workflow.

4.2.2 Authentication

Authentication in the frontend is handled in the `LoginPage` component in the `userLogin` directory. This component handles both user login and registrations. The interface is built using the Material UI (MUI), which provides a dark theme and responsive layout for all elements [27]

The component maintains a local state for all user inputs, including name, email, and password, using the `useState` hook. A toggle `isSignUp` determines if the form is in registration or login mode, which allows the conditional render of the "Full Name" field needed in registration. This avoids repeated logic and keeps all authentication functionality in a single component.

Client validation is done before any request is sent to the back end. Email fields are checked for correct formatting, and passwords must meet a minimum length requirement. Invalid inputs trigger error messages through MUI’s `error` and `helperText` properties, giving immediate responses to the user.

When the form is submitted, `registerUser()` or `loginUser()` is called from `api.js`. These functions send JSON-encoded POST requests to the back end endpoints. The `api.js` script performs error handling, checking the HTTP status of each response, and returning error messages when required. An example of the login request is shown below:

```
const response = await fetch(`${API_URL}/user/auth/login`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password })
});
```

If authentication is successful, the back end returns a JWT, which is then stored in the browser’s `localStorage`:

```
localStorage.setItem("token", data.access_token);
```

This follows the style of the back end’s JWT authentication style and ensures that the token persists across pages. Once stored, the user is redirected to the prediction history page through React Router’s `useNavigate` hook.

All protected requests automatically include the stored token through the API’s helper functions. For example, the prediction history request attaches the token in the `Authorization` header:

```
const token = localStorage.getItem("token");

const response = await fetch(`${API_URL}/api/predictions`, {
  headers: { Authorization: `Bearer ${token}` }
});
```

This ensures only authorised users can access protected endpoints. The same pattern is used for image uploads, where the token is included alongside the `FormData` payload.

Error handling is integrated into both components and the `api.js` script. If the back end returns an error, the `api.js` functions extract the error messages and throw an exception. The `LoginPage` component captures the exception and displays it to the user.

A screenshot of the Authentication interface is included in Figure 4.10.

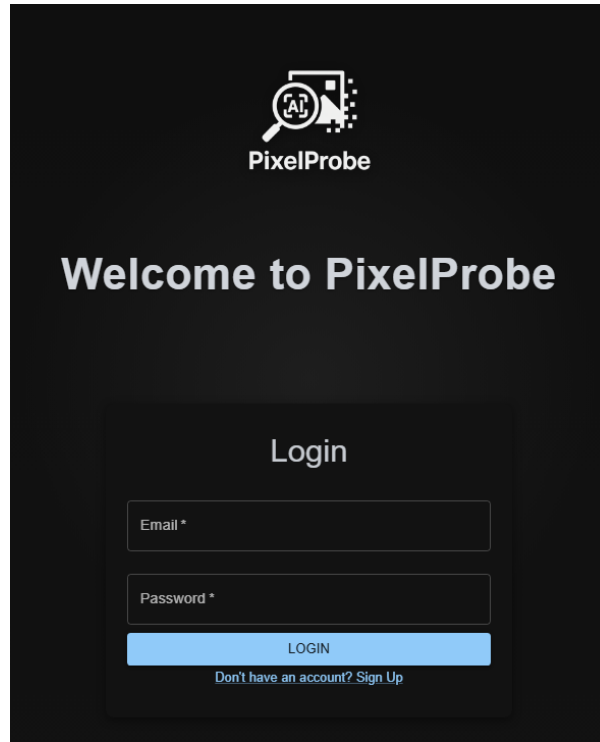


Figure 4.10: PixelProbe Login Screen

4.2.3 Image Upload

The image upload workflow is implemented in the `ImageUpload` directory, which contains the `ImageUpload` component which handles image selection, local preview of the image, submitting it to the back end and then returning a prediction result. This component mirrors the back end inference pipeline.

The component handles multiple pieces of local state through the `useState` hook, including selecting a file, a preview URL, the prediction result, loading status and any errors. When an image is selected the `handleChange()` function stores the file object and previews the image through `URL.createObjectURL()`. This allows the user to view the image before submission.

Once the image is selected, the user can use the `PredictionButton` component to complete inference. The `handlePredict()` function is executed, and calls the `createPred()` function from `api.js`. This function creates a `FormData` object, updates the selected file, and sends a POST request to the back end’s `uploadFile` endpoint as shown below:

```
const formData = new FormData();
formData.append("file", file);

const response = await fetch(`${API_URL}/api/uploadfile`, {
  method: "POST",
  headers: { Authorization: `Bearer ${token}` },
  body: formData
});
```

The request includes the user’s JWT in the `Authorization` header, ensuring only verified users can submit images for prediction. During the process, loading state is set to disable the prediction button and return feedback.

If the back end returns a successful response, the prediction object is stored in local state and displayed beside the image preview. The result includes the predicted class and confidence score. The confidence value is visualised using the `react-gauge-component` library, which renders a colour coded gauge showing the model’s confidence in its prediction. The gauge is configured with three segments (red, yellow, and green) to represent low, medium and high levels of confidence.

Error handling is integrated into the component. If the back end returns an error, the `createPred()` function throws an exception containing the error message. This is then displayed within the `ImageUpload` component back to the user.

A screenshot of the Upload interface is included in Figure 4.11

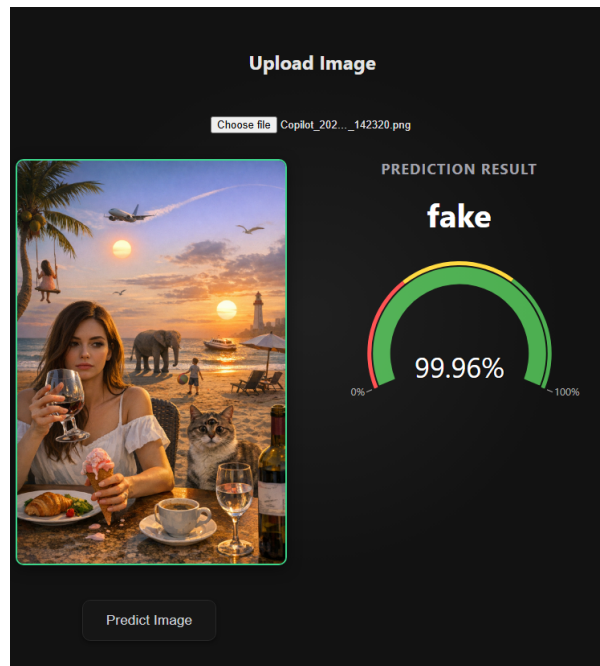


Figure 4.11: Image upload interface

4.2.4 Prediction History

The prediction history functionality is implemented in the `imageGallery` directory, which contains the components that retrieve and display all previous predictions associated with the current verified user.

The component maintains two pieces of state through the `useState` hook: the prediction dataset and a loading flag. When the component mounts, the `useEffect` hook triggers asynchronous data retrieval. This calls the `fetchPreds()` function from `api.js`, which sends an authenticated GET request to the back end.

```
const response = await fetch(`${API_URL}/api/predictions`, {
  headers: {Authorization: `Bearer ${token}`},
});
```

If the request is successful, the returned list of prediction objects are stored in local state and loading is disabled. If an error occurs, the component logs the error and terminates loading state, ensuring the interface remains responsive.

Each entry is rendered as a gallery card containing the image, and its metadata. Images are retrieved directly from the project’s S3 bucket using the filename returned in the back end. The component constructs the full S3 URL using a predefined base path:

```
const imageUrl = `${S3_BASE_URL}${item.image_name}`;
```

This allows the user to view each uploaded image alongside its classification. The card displays the prediction, confidence and timestamp.

Error handling is integrated into the component and the `api.js` script. If the back end returns an authentication error, the `fetchPreds()` function throws an exception, this error is then logged to the console, and results will not display until a user is logged in and verified.

A screenshot of the prediction history interface is included in Figure 4.12

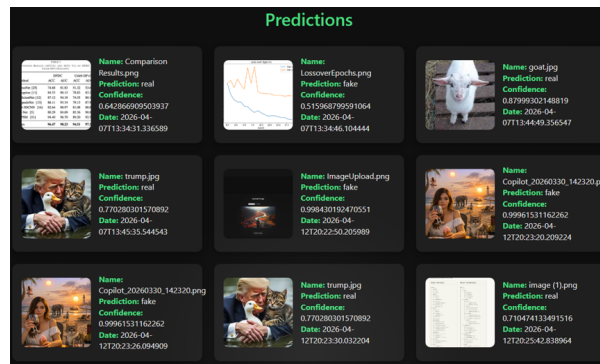


Figure 4.12: Prediction History Interface

4.2.5 User Interface (UI)

The user interface is implemented through a mix of React components, custom CSS, and select Material UI elements. The UI is designed to provide a clean, responsive experience across all features of the system. The interface follows a modular structure with each element encapsulated within its own component, ensuring maintainability and consistency throughout.

Navigation in the application is handled through the `Navbar` component. This component renders a static top navigation bar containing links to the Home, Image Upload and History pages. Navigation is done through React Router’s `Link` component, which allows client-side routing without page reloads. The navigation bar also shows the PixelProbe logo and is styled in `navbar.css` to maintain appearance across pages.

The routing structure is defined in `App.js`. This file configures the application’s layout using React Router’s `BrowserRouter`, `Routes` and `Route` components. The `Navbar` is rendered at the top of each page, and the main content changes dynamically based on the route. A simple version of the routing structure is as shown below:

```
<Routes>
  <Route path="/" element={<Homepage />} />
  <Route path="/predictions" element={<ImageUpload />} />
  <Route path="/history" element={<MyGallery />} />
</Routes>
```

The homepage contains the PixelProbe logo, a welcome message and the Login/Registration interface. This design ensures users are immediately presented with the login or registration form, and returning users can easily navigate to the upload and history pages. The layout is styled in `App.css`

Styling across the application is done through a combination of global CSS rules and component specific style sheets. Global styles in `App.css` and `index.css` define the overall colour scheme and layout while the component specific CSS files deal with design choices specific to each component. This approach ensures consistency in the visual design of the application.

A screenshot of the home screen is included in Figure 4.13

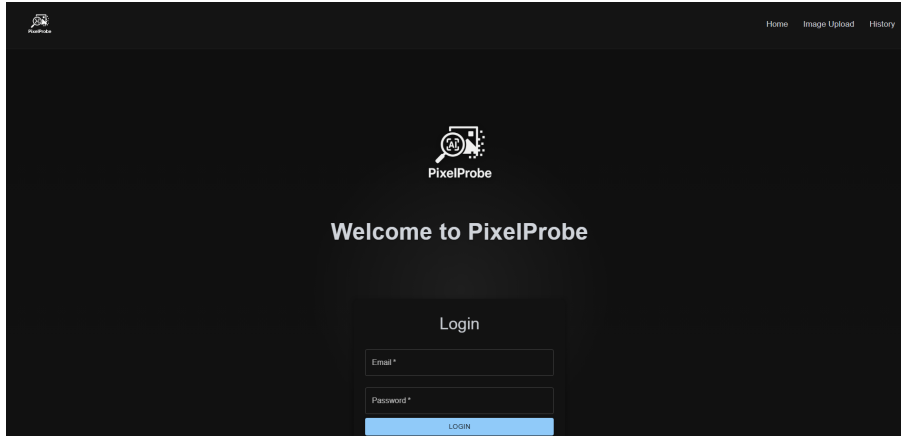


Figure 4.13: Main UI Screen

4.2.6 API integration

All communication between the frontend and back end is completed through `api.js`. This file defines helper functions that abstracts the details of HTTP request, ensuring that components focus on user interaction and presentation logic instead of networking operations.

The script defines the URL for all API requests. Each function creates a request to a specific back end endpoint, attaching the required headers and payloads. For protected routes, JWT is automatically retrieved from `localStorage` and included in the header. This ensures authentication is consistent across all components.

The `fetchPreds()` function pull the authenticated user’s prediction history by sending a Get request to the `api/predictions` endpoint. The function attaches the JWT and returns a parsed JSON response as shown below:

```
export async function fetchPreds() {
  const token = localStorage.getItem("token");

  const response = await fetch(`${API_URL}/api/predictions`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  if (!response.ok) {
    throw new Error("Failed to fetch predictions");
  }

  return response.json();}
```

If a request fails, the function throws an error, which is handles by the component calling the function. This allows the frontend to display the related error messages to the user when they occur.

The `createPred()` function handles image uploads for prediction. It creates a `FormData` object, appends the selected file, and sends a POST request to `/api/uploadFile`. Since the file uploads need multipart encoding, the browser assigns the appropriate `Content-type` header. The JWT is attached in the `Authorization` header as shown below:

```
export async function createPred(file) {
  const token = localStorage.getItem("token");

  const formData = new FormData();
  formData.append("file", file);

  const response = await fetch(`${API_URL}/api/uploadfile`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${token}`,
    },
    body: formData,
  });

  if (!response.ok) {
    const errorText = await response.text();
    throw new Error(`Failed to create prediction: ${errorText}`);
  }

  return response.json();
}
```

If an error is returned from the back end, the function extracts the error message and throws an exception which can then be displayed by the calling component.

The authentication functions `registerUser()` and `loginUser()` send JSON-encoded POST requests to the back end’s registration and login endpoints. Both check the HTTP status of the response and throw errors when authentication fails. Successful responses return a JWT, which is then stored in `localStorage` and can be reused across different requests.

Through separating the back end communication into `api.js`, the frontend keeps a separation between data handling and the UI logic. This design improves maintainability and ensures any changes to API endpoints can be implemented without modifying individual components.

4.3 Machine Learning

4.3.1 Dataset Preparation

The preparation of data is a critical component of any machine learning system, the quality and consistency of the input data directly affects the performance and generalisation ability of the final model. In this project, multiple datasets were used to train various model versions, these included `CIFAKE`, `Dalle_image`, and `GenImage`. Each model version was able to learn from their specific input dataset. Training different iterations of the model allows for controlled comparisons between datasets and their ability to generalise.

Each dataset was organised into the same two class format required by Tensorflow's `image_dataset_from_directory()` loader. Each dataset had the following structure:

```
dataset_name/
  train/
    real/
      img001.jpg
      img002.jpg
      ...
    fake/
      imgA.jpg
      imgB.jpg
      ...
  test/
    real/
      img001.jpg
      img002.jpg
      ...
    fake/
      imgA.jpg
      imgB.jpg
      ...
```

This ensures that all datasets can be loaded using the same code during processing.

Dataset Loading

Each dataset was loaded through the following TensorFlow loading pipeline

```
IMG_SIZE = 224
BATCH_SIZE = 64
DATA_DIR = "BE/GenImage"

train_dir = f"{DATA_DIR}/train"
test_dir = f"{DATA_DIR}/test"

ds_train = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode="int"
)

ds_test = tf.keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode="int"
)
```

Using this structure for all datasets ensured each model was trained under the same conditions, regardless of what dataset was used. This allows for fair comparisons between models.

A fourth dataset was kept separate from all training datasets and was not used to train any model versions. This dataset was solely used as a control dataset in cross-dataset generalisation comparisons.

4.3.2 Preprocessing and Data Augmentation

Once the datasets are all prepared, each set goes through a standardised preprocessing and augmentation pipeline. This is designed to normalise image inputs, improve model robustness and reduce over-fitting. Each dataset goes through the same preprocessing to ensure consistency across models for comparison.

Image Resizing and Normalisation

All images are resized to 224×224 pixels to match the requirements of EfficientNetV2. This step is done through the TensorFlow `Resizing` layer, which ensures each image sent to the model has a uniform resolution. After resizing, each image is normalised using `preprocess_input` from the EfficientNetV2 preprocessing suite. This scales pixel values to a range expected by the pre-trained ImageNet weights, ensuring alignment with the model’s original training distribution.

```
x = Resizing(IMG_SIZE, IMG_SIZE)(inputs)
x = preprocess_input(x)
```

Normalisation is complete after augmentation to ensure augmented images are still compatible with EfficientNet.

Data Augmentation

To improve generalisation, random augmentations are applied to the training dataset. This introduces randomness to the training process by applying geometric and photometric transformations. The augmentation pipeline is done using a `keras.Sequential` model with the following layers:

- **RandomRotation(0.15):** rotates images by up to 15% of 360° , improving rotational invariance.
- **RandomTranslation(0.1, 0.1):** shifts images horizontally and vertically by up to 10%, helping the model learn positional robustness.
- **RandomFlip():** applies horizontal flips, simulating mirrored versions of the same object.
- **RandomContrast(0.1):** adjusts image contrast by $\pm 10\%$, improving resilience to lighting variations.

The augmentation model is defined as:

```
img_augmentation_layers = keras.Sequential([
    layers.RandomRotation(0.15),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomFlip(),
    layers.RandomContrast(0.1),
], name="img_augmentation")
```

This pipeline is only applied during training and is disabled during validation and testing to ensure the evaluation metrics are realistic.

Model Integration

The augmentation module is inserted into the model’s architecture after the input layer as shown below:

```
inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = Resizing(IMG_SIZE, IMG_SIZE)(inputs)
x = img_augmentation_layers(x)
x = preprocess_input(x)
```

The preprocessing and augmentation pipeline ensures all datasets are in a consistent, normalised format suitable for EfficientNet training. Augmentation introduces controlled variability that aids in improving generalisation particularly on the control test set.

4.3.3 Model Architecture

PixelProbe is built using a hybrid architecture combining the pre-trained EfficientNet backbone with a custom lightweight multi-head self-attention module (LiteMHSA). This design combines the representational strength of modern CNNs while including attention based mechanism to capture long-range spatial dependencies. The architecture is implemented in `training.py` and is shared across all dataset-specific model versions.

EfficientNet Backbone

The core of the module is the EfficientNet architecture, a modern CNN pre-trained on ImageNet. The backbone is loaded with `include_top=False` to remove the original classification head allowing a custom head to be tailored to the binary classification task.

```
base_model = EfficientNetV2B2(
    include_top=False,
    weights='imagenet',
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False
```

Freezing the backbone in the first training phase preserves the pre-trained features of the model and reduces instability when training new layers.

LiteMHSA Attention Layer

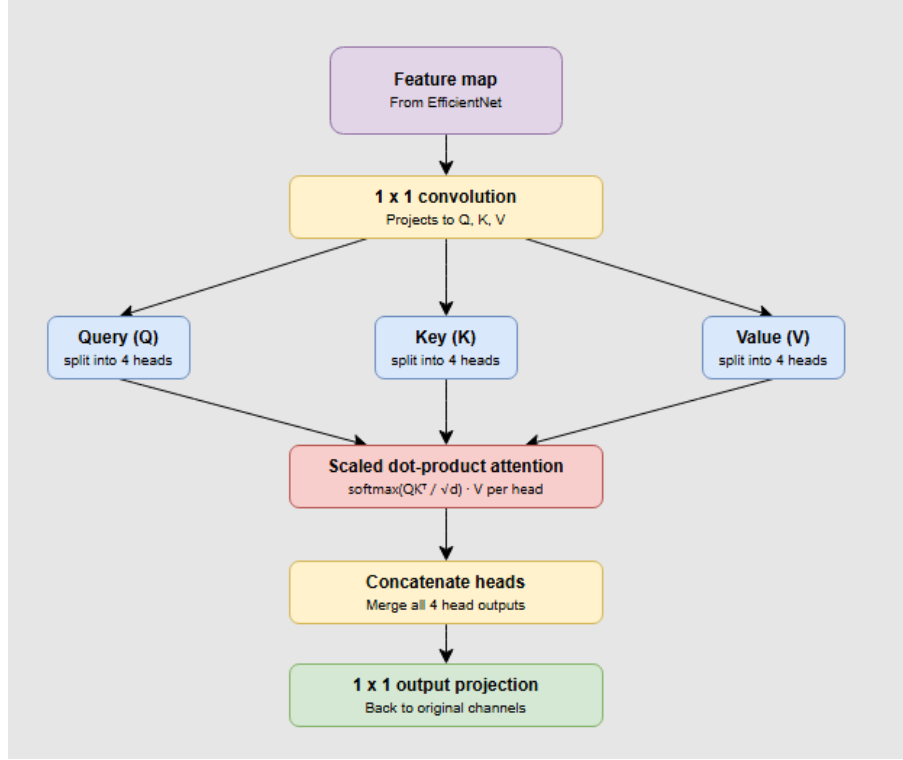


Figure 4.14: Internal structure of the LiteMHSA attention mechanism.

To improve the model’s ability to detect subtle artefacts, a custom lightweight multi-head self-attention layer (LiteMHSA) was inserted after the EfficientNet feature extractor. This layer operated on spatial feature maps created by the backbone and computes attention across the whole image, allowing the model to capture global relationships that convolutional kernels may miss.

The LiteMHSA layer performs the following:

- Splits the feature map into query, key and value using a 1×1 convolution
- Splits each tensor into multiple attention heads
- Computes scaled dot-product attention
- Recombines the attended features and projects them back to the original channel dimensions.

The implementation is defines as a custom Keras layer:

```
x = LiteMHSA(dim=c, heads=4, reduction=4)(x)
```

This module add minimal computational overhead while hugely improving the model’s ability to distinguish between real and AI generated images.

Classification Head

After the attention layer, the model adds a global average pooling layer to collapse spatial dimensions, it then add dropout for regularisation and a dense soft max layer for classification:

```
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

NUM_Classes = 2 corresponds to the real and fake classes..
```

Two-Phase Training

To stabilise training and maximise performance, the model is trained in two phases:

1. **Phase 1 - Train classification head only:** The EfficientNet backbone is froze, allowing the new layers (augmentation, attention, pooling, dropout, dense) to learn without affecting the pre-trained features.
2. **Phase 2 - Fine-tune the top 20 layers:** After the head converges, the top 20 convolutional layers are unfrozen using the `unfreeze_model` function. Batch normalisation layers are kept frozen to prevent instability.

```
def unfreeze_model(base_model, num_layers=20):
    for layer in base_model.layers[-num_layers:]:
        if not isinstance(layer, layers.BatchNormalization):
            layer.trainable = True
```

Fine tuning is done with a reduced learning rate (1×10^{-5}) to avoid forgetting of pre-trained features.

This two-phase strategy improves generalisation, especially when training the model on smaller datasets.

Summary

The complete architecture is summarised in Figure 4.15

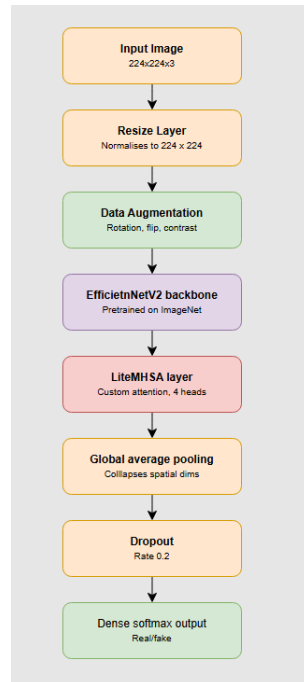


Figure 4.15: Complete model architecture

The hybrid architecture of the model mixes the strengths of convolutional and attention based feature extraction, which results in a more robust classifier capable of detecting subtle artifacts across different datasets.

4.3.4 Training Pipeline

The training pipeline of PixelProbe’s models follows a two-phase strategy to stabilise optimisation, preserve pre-trained features and maximise performance across datasets. It consists of dataset loading, model compilation, training, fine-tuning, check-pointing and training history exports.

Dataset Loading and Batching

The dataset is loaded through using `image_dataset_from_directory()`, which automatically handles batching, shuffling and label assignment. A batch size of 64 is used to balance memory constraints with training throughput. The validation set is loaded in the same way, ensuring consistent preprocessing.

```
ds_train = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode="int"
)
```

Phase 1: Training the Classification Head

The first phase only trains newly added layers such as augmentation, LiteMHSA, pooling etc. with the EfficientNet backbone frozen. This prevents pre-trained convolutional filters from being forgotten before the classification head stabilised. The model is compiled with the Adam optimizer and a standard learning rate:

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Training is done over 15 epoch with early stopping and model check-pointing active:

```
history1 = model.fit(
    ds_train,
    validation_data=ds_test,
    epochs=15,
    callbacks=my_callbacks
)
```

This phase allows the classifier adapt to dataset specific patterns using the pre-trained features from EfficientNet.

Phase 2: Fine-Tuning

After the classification head converges, the top 20 layers are unfrozen to allow fine-tuning. Batch normalisation layers remain frozen to avoid destabilisation.

```
unfreeze_model(base_model, num_layers=20)
```

Fine tuning is performed with a much lower learning rate of (1×10^{-5}) to stop pre-trained features being forgotten.

```
model.compile(
    optimizer=keras.optimizers.Adam(1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

The model is then trained for an additional 30 epochs with early stopping and check-pointing:

```
history2 = model.fit(
    ds_train,
    validation_data=ds_test,
    epochs=30,
    callbacks=my_callbacks
)
```

This two phase approach ensures the model learns high level decision boundaries prior to refining low level convolution features to detect generative artefacts better.

Callbacks

Three callbacks are used to stabilise training and save the best performing model.

- **EarlyStopping:** Stops training when validation loss stops improving
- **ModelCheckpoint:** Saves the best model to BE/model/best_model.keras
- **TensorBoard:** Logs training metrics.

Training History

After both phases are complete, the histories are joined together and saved as a JSON file:

```
combined_history = {
    k: history1.history[k] + history2.history[k]
    for k in history1.history
}
with open("BE/model/historyB2_GenImage_V3.json", "w") as f:
    json.dump(combined_history, f)
```

This file is then used by `analysis.py` to create training curves.

Summary

The training pipeline is a combination of transfer learning, and fine-tuning to produce a robust classifier that is able to detect subtle artifacts. By using the same pipeline for each model iteration, there is a fair comparison between the generalisation between model versions.

4.3.5 Model Export and Integration

Once the training process is complete, the final step is to export the trained model. This ensures the model can be loaded and used across the PixelProbe system.

Model Saving

Once all training steps are complete, the model is saved as a `.keras` file using `model.save()`. This format saves the full models architecture, weight, optimiser and custom layers allowing storage and deployment.

```
model.save("BE/model/pixelProbeB2_GenImage_V3.keras")
```

The `BE/model` directory is created if it does not exist and all model `.keras` files are stored here.

Handling Custom layers

Since the model includes a custom MHSA layer, Tensorflow must be told that this layer exists when loading the model during inference. This is done by passing the layer through the `custom_objects` argument:

```
model = tf.keras.models.load_model(  
    model_path,  
    custom_objects={"LiteMHSA": LiteMHSA}  
)
```

Without including this step, TensorFlow would be unable to reconstruct the model, resulting in deserialisation. Including this layer ensures the model remains portable and compatible with the back end scripts.

Back end Integration

The exported model is integrated into the FastAPI back end, where it is loaded once at startup and used to complete inference on uploaded images. The back end applies the same preprocessing before passing images to the model. Once inference is complete the back end returns the predicted class and confidence score to the frontend where it is displayed to the user.

Summary

The model export and back end integration process ensures that every model version is saved, reproducible and compatible with the back end inference pipeline.

4.3.6 Evaluation

Once the trained model is exported and saved, the final stage is to evaluate the model’s performance in training and on unseen data. This is done in `analysis.py`, which completes three tasks:

- Loads the trained model and its history
- Generates predictions on the test set
- Computes metrics and visualises accuracy scores and confusion matrices.

This ensures all model versions are evaluated under the same conditions

Model Loading

The evaluation script loads the trained model from BE/model, It includes the custom LiteMHSA attention layer, registering it in the custom_objects argument to ensure correct deserialisation:

```
model = tf.keras.models.load_model(
    model_path,
    custom_objects={"LiteMHSA": LiteMHSA}
)
```

This reconstructs the full model, including the EfficientNetV2 backbone, the LiteMHSA layer and the classification head.

Loading the Test Set

The test dataset is stored in its respective training dataset as a subdirectory and is separated from the training directory. This ensures the validation reflects the true performance of the model, rather than memorising training images. The database is loaded using the image_dataset_from_directory() function with shuffle=False to preserve ordering:

```
ds_test = tf.keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode="int",
    shuffle=False
)
```

The class names are automatically taken and used when constructing the confusion matrix.

Training History Visualisation

The training history file is loaded and generates two plots:

- Training vs. validation loss over epochs
- Training vs. validation accuracy over epochs

These plots give insight into the behaviour, over-fitting and effectiveness of the training strategy. These plots are saved to BE/plots.

Accuracy

Predictions are done in batches using a progress bar from the tqdm library. The script collects all ground-truth labels and predicted class indices, then computes a test accuracy using accuracy_score() from scikit-learn:

```
y_true = np.concatenate([y.numpy() for _, y in ds_test], axis=0)
y_pred = []
for batch, _ in tqdm(ds_test, desc="Predicting", unit="batch"):
    preds = model.predict(batch, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
```

This ensures that predictions are evaluated in the same order as the dataset’s labels, ensuring accurate computation.

Confusion Matrix

To get a detailed breakdown of the model’s performance, a confusion matrix is created using `confusion_matrix()` and visualised with `ConfusionMatrixDisplay`. The matrix highlights the class-specific accuracies and can indicate if it is more prone to false positives or false negatives:

```
cm = confusion_matrix(y_true, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap=plt.cm.Blues, xticks_rotation=45)
```

The confusion matrix is also saved in `BE/plots`.

Summary

The evaluation used in this project provides a consistent method for assessing each version of the model trained across different datasets. By separating training and evaluation into different scripts, the system ensures performance is based on unseen data and visualisations are standardised.

4.3.7 Overall Summary

A standardised preprocessing pipeline was applied across all datasets, including geometric and photometric transformations to help generalise. All images are resized to 224×224 pixels and normalised using EfficientNet preprocessing to ensure compatibility.

The model’s architecture combined EfficientNet’s feature extractor with a custom MHSA layer and a lightweight classification head. This design allowed the model to capture both local and global spatial relationships. A two phase training strategy was used to first train the classification head with a frozen backbone, followed by fine-tuning upper layers of EfficientNet.

After training, the model and its training history are saved for future deployment. The backend loads the model at startup, and applies the same preprocessing as used during training. `Analysis.py` is used to assess the performance of each model version, generating training curve plots and confusion matrices.

Overall, this implementation provides a modular and robust workflow that supports multiple datasets and easy deployment with the PixelProbe system.

4.4 Results

Four iterations of PixelProbe were trained based on the CIFAKE, DALLE_Images, GenImage and a hybrid of the GenImage and Places365 datasets. Each model was evaluated on their own dataset’s test set, each of the other four datasets, and a separate test set that acted as a control. The results are shown in training curves, accuracy scores, and confusion matrices.

4.4.1 Training and Validation Performance

The training pipeline created a history file detailing all information collected during training. The `analysis.py` scripts created plots of training and validation loss and accuracy across all epochs.

Figure 4.16 shows the loss curves for one of the model versions. the training loss decreases steadily throughout, while the validation loss stabilises once fine-tuning begins. This behaviour indicates the two phase strategy correctly prevents over fitting while the model refines high level features in fine tuning.

Figure 4.17 shows the accuracy curves. The model shows quick improvement in the initial head training phase, with some smaller gradual gains during fine tuning. This is consistent across all datasets.

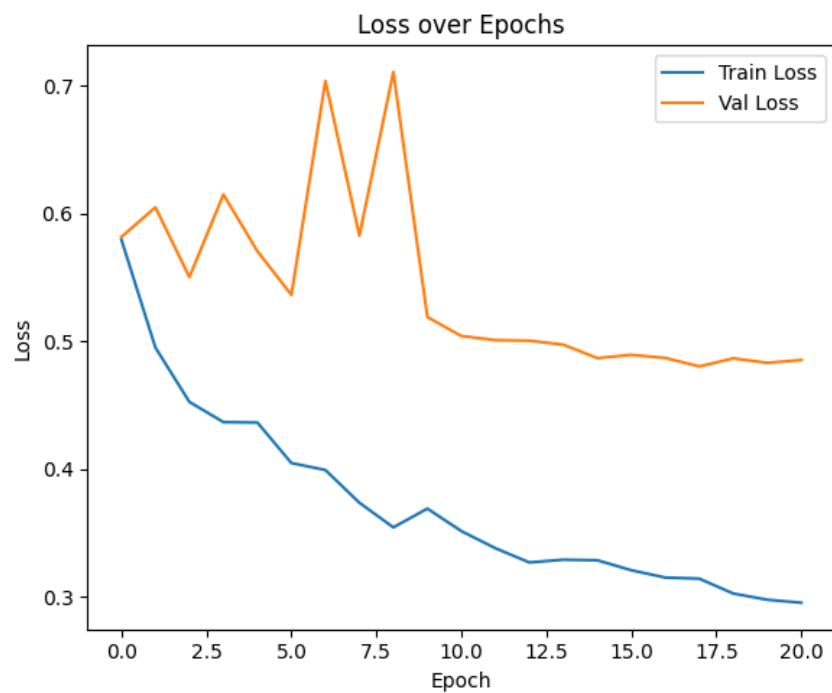


Figure 4.16: Training and validation loss across all epochs.

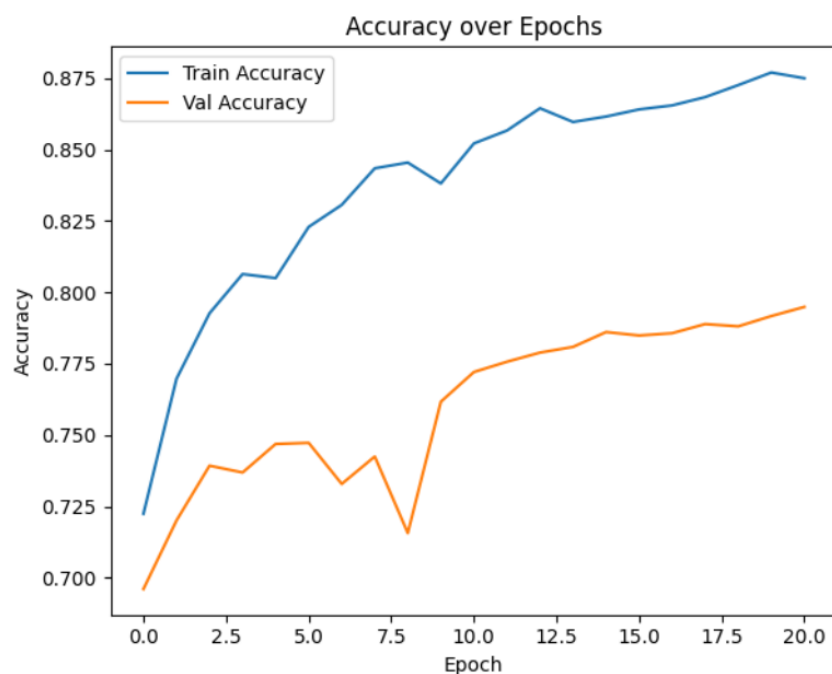


Figure 4.17: Training and validation accuracy across all epochs.

4.4.2 Test Accuracy

Each model was evaluated across all other datasets, including the control `testSet`. The test accuracy was computed using the `accuracy_score()` function from `scikit-learn`. Table 4.1 summarises the results for all three dataset-specific models.

Model	Train Set	ZHANG	CIFAKE	GEN/P365	GHAI
PixelProbeB0	ZHANG	86	50	58	70
PixelProbeB1_V3	ZHANG	94	51	57	<u>78</u>
PixelProbeB1CFV2	CIFAKE	44	95	51	26
PixelProbeB1GI	GenImage	52	<u>61</u>	80	77
PixelProbeB2GI	GenImage	50	56	<u>79</u>	75
PixelProbeB2GIV3	GenImage/Places365	<u>56</u>	54	78	73

Table 4.1: Test accuracy for each dataset-specific model version.

The GenImage trained model typically achieved the highest accuracy due to the larger diversity of the dataset, while Dalle_Images performed slightly less due to the smaller dataset size. CIFAKE performed well on stylised AI images but did not generalise well on photorealistic samples.

4.4.3 Confusion Matrix Analysis

To gain a better understanding of the model’s behaviour, confusion matrices were created for each model version. Figure 4.18 shows an example confusion matrix.

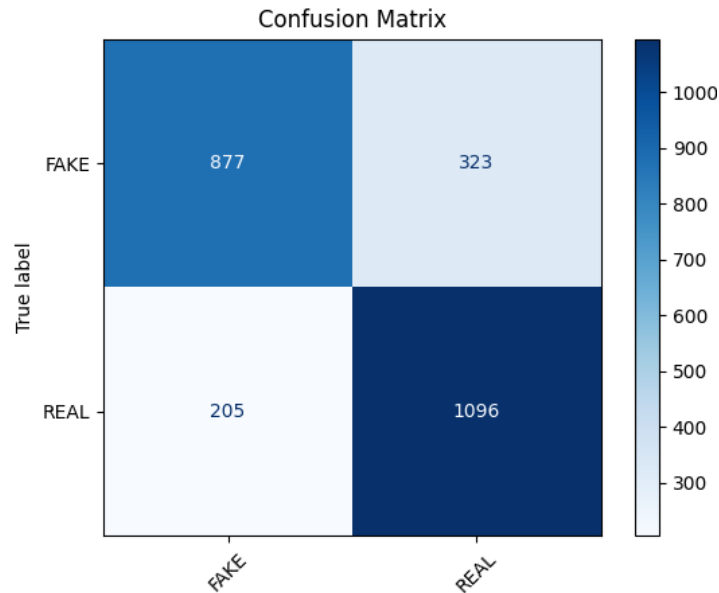


Figure 4.18: Confusion matrix for one of the trained model versions.

The confusion matrices revealed common misclassification patterns in AI-image detection: [28]

- **Real → Fake misclassification** may be due to compression artifacts, noise, or unusual textures in real images, which are known to affect detectors.
- **Fake → Real misclassification** is likely due to more photorealistic images, which have been shown to have fewer detectable artifacts compared to older generators.

4.4.4 Cross-Dataset Comparison

Training with different datasets allowed for a complete comparison of how each dataset influences performance. The following was observed

- **CIFAKE:** Performs well on stylised synthetic images but generalises badly on photorealistic images
- **Dalle_Image:** Achieves moderate performance due to dataset size
- **GenImage:** Achieves the strongest overall performance and best generalisation to the control test set.

4.4.5 Summary of Results

In general, the results show that combining EfficientNet feature extraction with a LiteMHSA attention mechanism produces reliable results across all datasets. The training and validation curves showed smooth convergence consistently. The two phase training method did well to reduce over-fitting and enabled the model to better refine higher-level features in fine-tuning.

Cross dataset analysis showed clear differences in generalisation based on the training dataset. Models trained on CIFAKE performed well on stylised images but failed to yield great results on more photorealistic ones, this confirmed that CIFAKE is not suited when trying to improve generalisation.

Models trained on GenImage and the hybrid GenImage/Places365 datasets achieved the best overall performance across all external datasets, including the control test set, this indicated a broader diversity in the dataset leads to more transferable feature learning.

The confusion matrices highlighted classification patterns across all the models. Real images that contained compression artifacts or unusual textures were labelled fake, while more photorealistic synthetic images, in particular from diffusion generated were occasionally misclassified as real. This suggest that the model is sensitive to texture cues.

Overall, the results show a more diverse dataset is the strongest in cross dataset generalisation, and that the hybrid EfficientNet/MHSA architecture is more than capable of learning generalisable features when trained on sufficient data. The GenImage models consistently performed the best both in their own dataset and outside, making them the most reliable of those tested.

5 Conclusion

The initial aim of this project was to design a complete end-to-end system capable of detecting AI generated images through machine learning techniques and a fully integrated software architecture.

The machine learning model was the core of the system. Multiple datasets were prepared and processed independently, and used to develop multiple dataset specific models. This resulted in a controlled environment for evaluating how each dataset affected overall performance. A standard preprocessing pipeline was used for all model versions, and the evolution to a EfficientNet/LiteMHSA hybrid architecture showed strong improvements and a balance between pre trained features and attention based global reasoning/

The two phase strategy was proven effective. Training the classification head first allowed the model to stabilise before fine-tuning upper layers of the EfficientNet backbone. This utilised transfer learning efficiently and produced capable models. The evaluation script provided clear insights through accuracy metrics, training curves, and confusion matrices.

The project delivered a fully functional application. The back end, built in FastAPI, loads the trained model at startup and performs inference using the same preprocessing as in training. The frontend provides a clean, reactive UI that allows users to login/register, upload files for inference and view a personalised history of predictions. This shows how a machine learning model can be deployed in a practical way.

Overall, PixelProbe achieved its goal of detecting AI Generated images through a robust machine learning pipeline. The system shows a strong ability across multiple datasets, a modular architecture that supports future scalability, and a deployment ready design for practical use.

6 Future Work

PixelProbe successfully provides a functional system for detecting AI generated images with positive accuracy, however, there are several ways in which it can be expanded and improve its real world applicability. Two main directions for expansion are frame-by-frame video detection and the deployment of a mobile application.

6.1 Video Frame-by-Frame Detection

A natural extension to the current system would be to implement the ability to analyse video content. As generative models become more capable of creating synthetic videos, frame by frame detection will become essential for identifying AI generated sections of longer media. This would involve implementing several new components.

- **Frame Extraction:** Videos would be split into individual frames and samples in intervals to balance accuracy with computational cost.
- **Video Level Reporting:** Results could be collected to produce timeline of real and fake segments to allow users to identify edited portions of a video

This extension would broaden the system’s applicability, particularly in content moderation.

6.2 Mobile App Deployment

Another direction to go in the future would be to deploy PixelProbe as a mobile application. A mobile version would make the system more accessible and allow users to perform real time detection on their devices. There are several considerations that would need to be taken to do this:

- **Model Optimization:** The current architecture may be too large for mobile devices. Conversion to TensorFlow Lite or introducing quantization or pruning would be needed to reduce the model size and latency at inference.
- **On-Device vs, Cloud Inference:** On-Device inference offers privacy and offline capabilities, while cloud inference provides access to larger models.
- **Camera Integration:** The app could allow users to capture images directly or analyse photos from their gallery, improving the user experience.
- **Cross platform support:** Producing the app on React Native or Flutter would allow deployment on both Android and iOS with a shared codebase.

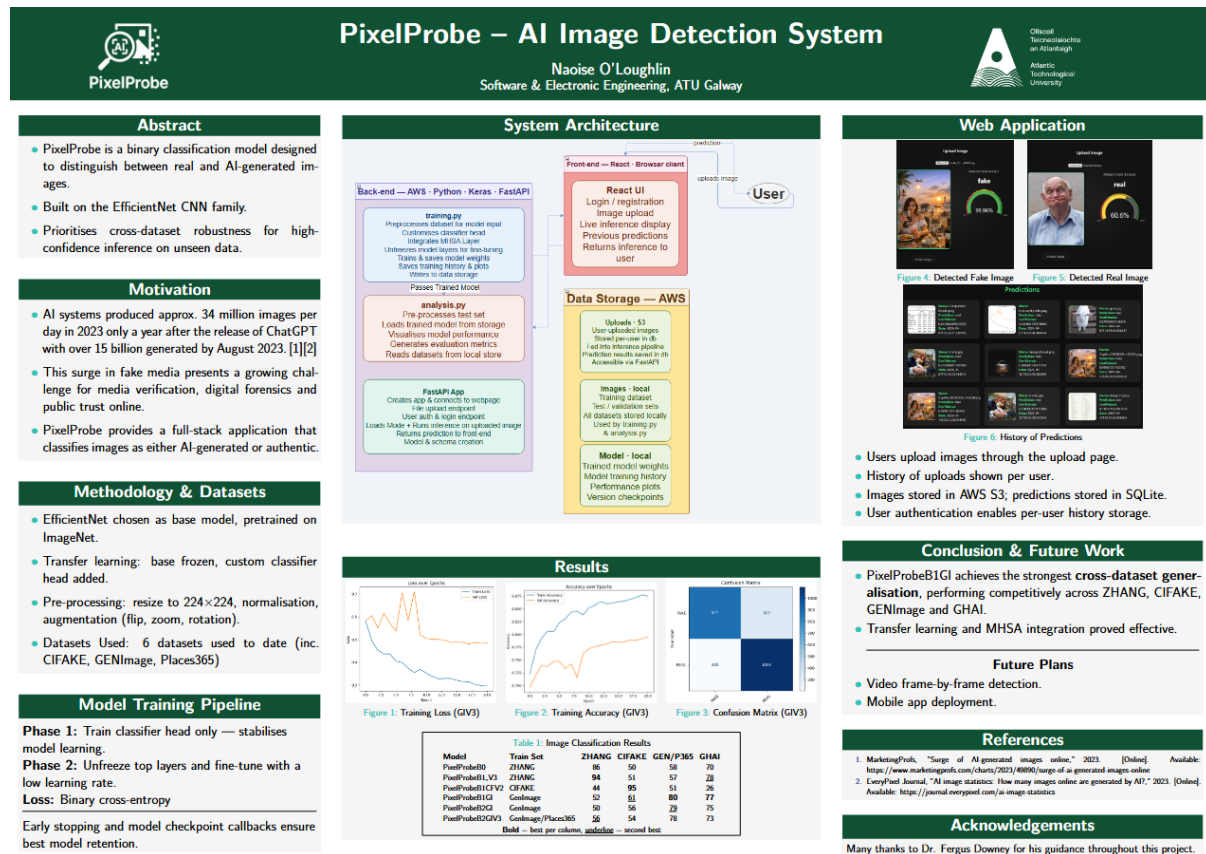
Mobile deployment would hugely expand the accessibility of PixelProbe and allow real-time detection in everyday scenarios, such as on social media or messaging platforms.

6.3 Summary

Both possible expansions would be meaningful additions to PixelProbe expanding it from a prototype system to a production ready tool. These improvements would enhance scalability and accessibility, ensuring the system remains relevant as generative technologies evolve

A Appendix - Additional Material

A.1 Project Poster



A.2 Training Curves & Confusion Matrices

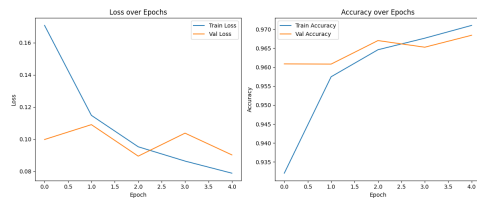


Figure A.1: B1_V3 Training curves

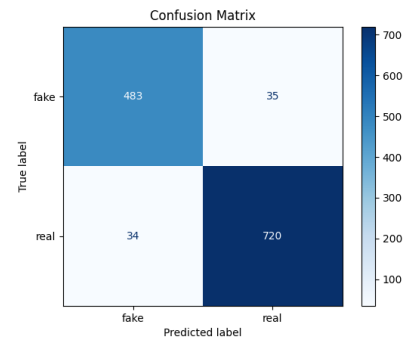


Figure A.2: B1_V3 Confusion Matrix

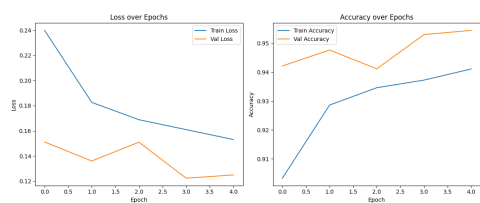


Figure A.3: B1_Cfv2 Training curves

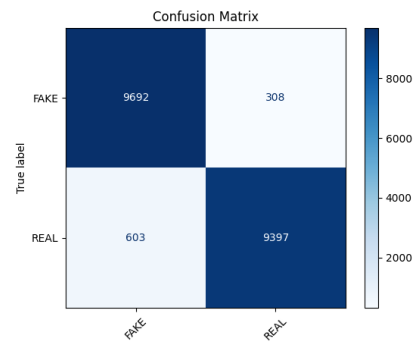


Figure A.4: B1_Cfv2 Confusion Matrix

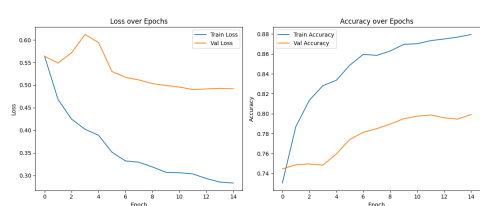


Figure A.5: B1_GI Training curves

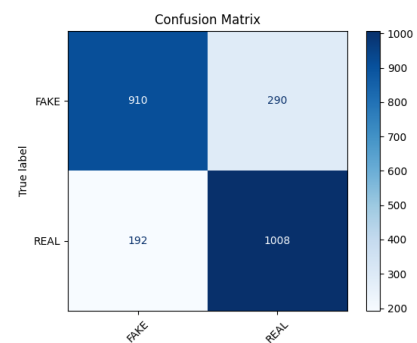


Figure A.6: B1_GI Confusion Matrix

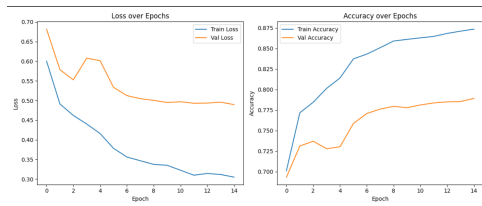


Figure A.7: B2_GI Training curves

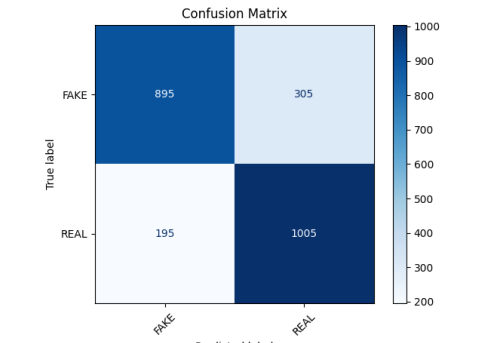


Figure A.8: B2_GI Confusion Matrix

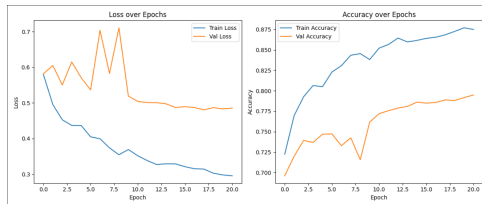


Figure A.9: B2_GI_v3 Training curves

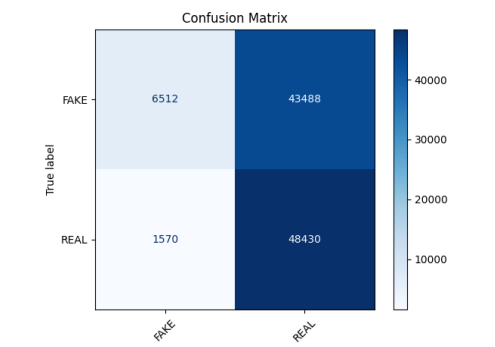


Figure A.10: B2_GI_v3 Confusion Matrix

A.3 LiteMHSA Class code

```

class LiteMHSA(layers.Layer):
    def __init__(self, dim, heads=4, reduction=4, **kwargs):
        super().__init__(**kwargs)
        self.dim = dim
        self.heads = heads
        self.reduction = reduction
        self.inner = dim // reduction

        self.qkv = layers.Conv2D(self.inner * 3, 1, padding="same")
        self.proj = layers.Conv2D(dim, 1, padding="same")

    def get_config(self):
        config = super().get_config()
        config.update({
            "dim": self.dim,
            "heads": self.heads,
            "reduction": self.reduction
        })
        return config

    def call(self, x):
        B = tf.shape(x)[0]
        H, W, C = tf.shape(x)[1], tf.shape(x)[2], tf.shape(x)[3]

        qkv = self.qkv(x)
        q, k, v = tf.split(qkv, 3, axis=-1)

        def reshape_heads(t):
            t = tf.reshape(t, [B, H * W, self.heads, self.inner // self.heads])
            return tf.transpose(t, [0, 2, 1, 3])

        q = reshape_heads(q)
        k = reshape_heads(k)
        v = reshape_heads(v)

        attn = tf.matmul(q, k, transpose_b=True)
        attn = attn / tf.math.sqrt(tf.cast(self.inner // self.heads, tf.float32))
        attn = tf.nn.softmax(attn, axis=-1)

        out = tf.matmul(attn, v)
        out = tf.transpose(out, [0, 2, 1, 3])
        out = tf.reshape(out, [B, H, W, self.inner])

        return self.proj(out)

```

Figure A.11: Full implementation of the LiteMHSA layer referenced in Section 4.3.3

Bibliography

- [1] Y. Yin, L. Huang, Y. Liu, and K. Huang. (2022) Diffgar: Model-agnostic restoration from generative artifacts using image-to-image diffusion models. arXiv. [Online]. Available: <https://arxiv.org/abs/2210.08573>
- [2] K. McDonald. Why grok restrictions won’t stop society’s latest ai scourge. RTÉ News. [Online]. Available: <https://www.rte.ie/news/primetime/2026/0118/1553516-grok-restrictions/>
- [3] K. R. Harris. (2024) Synthetic media detection, the wheel, and the burden of proof. Springer Nature. [Online]. Available: <https://link.springer.com/article/10.1007/s13347-024-00821-0>
- [4] S. Ramírez. (2024) Fastapi documentation. FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] T. Suslov. (2024) Uvicorn documentation. Uvicorn. [Online]. Available: <https://uvicorn.dev/>
- [6] A. W. Group. (2024) Asgi specification documentation. ASGI. [Online]. Available: <https://asgi.readthedocs.io/en/latest/>
- [7] D. R. Hipp. (2024) Sqlite documentation. SQLite. [Online]. Available: <https://sqlite.org/docs.html>
- [8] A. W. Services. (2024) Amazon s3 documentation. AWS. [Online]. Available: <https://docs.aws.amazon.com/s3/>
- [9] P. Team. (2024) Pydantic models and validation. Pydantic. [Online]. Available: <https://pydantic.dev/docs/validation/latest/concepts/models/>
- [10] S. Ramírez. (2024) Fastapi security – simple oauth2. FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/#code-to-get-the-username-and-password>
- [11] PyPI. (2024) bcrypt. Python Package Index. [Online]. Available: <https://pypi.org/project/bcrypt/>
- [12] A. W. Services. (2024) What is object-relational mapping (orm)? AWS. [Online]. Available: <https://aws.amazon.com/what-is/object-relational-mapping/>
- [13] M. P. Inc. (2024) React documentation. [Online]. Available: <https://react.dev/>
- [14] Meta Platforms Inc. (2024) Using the state hook. [Online]. Available: <https://react.dev/reference/react/useState>
- [15] M. W. Docs. (2024) Fetch api. Mozilla Developer Network. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [16] MDN Web Docs. (2024) Formdata. Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/FormData>
- [17] M. P. Inc. (2024) Jsx in depth. [Online]. Available: <https://react.dev/learn/writing-markup-with-jsx>

- [18] M. W. Docs. (2024) Javascript. Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [19] D. Contributors. (2024) Css reference. DevDocs. [Online]. Available: <https://devdocs.io/css/>
- [20] N. Contributors. (2024) Node.js api documentation. OpenJS Foundation. [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [21] R. R. Team. (2024) React router documentation. Remix Software. [Online]. Available: <https://reactrouter.com/home>
- [22] T. Zhang. (2023) Ai-generated images vs real images dataset. Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/tristanzhang32/ai-generated-images-vs-real-images>
- [23] Birdy654. (2023) Cifake: Real and ai-generated synthetic images. Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images>
- [24] R. Huang. (2023) Genimage subset detection dataset. Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/renhuang8/genimage-subset-detection>
- [25] B. K. Zhang. (2022) Places365 dataset. Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/benjaminkz/places365>
- [26] R. Ghai. (2023) Ai vs real images dataset. Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/rhythmghai/ai-vs-real-images-dataset>
- [27] MUI Team. (2026) Material ui: Getting started. MUI. [Online]. Available: <https://mui.com/material-ui/getting-started/>
- [28] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, “On the detection of synthetic images generated by diffusion models,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5. [Online]. Available: <https://arxiv.org/abs/2211.00680>